> Could you give me the kind of tasks that this kind of app will do. If dribble, Asana, jira, notion and microsoft projects could have a marriage and get a hybrid child with all the premium feature. what will this child be capable of doing, what tasks will it be capable of performing

The Hybrid App: "Dribbble × Asana × Jira × Notion × MS Project" — What It Would Do

Think of this as a **single workspace where visual design inspiration, project management, agile engineering, docs/wiki, and enterprise portfolio planning all live in one graph**. Here are the task categories it would perform:

1. Visual & Creative Work Management (Dribbble layer)

- Host **design shot galleries** and mood boards tied directly to project tasks
- **Visual task cards** — attach mockups, prototypes, and version histories inline
- **Design critique threads** with annotated screenshots (pin comments to pixels)
- **Asset library** with licensing, versioning, and usage tracking
- **Portfolio → project linking** (a shot becomes a deliverable, tracked to completion)
- **Brand/design-system hub** feeding tokens and components into dev tickets

2. Work & Task Management (Asana layer)

- **Multi-view tasks**: list, board, timeline, calendar, workload, Gantt — same data, switch anytime
- **Dependencies, milestones, and automated workflows** (if X done → assign Y)
- **Goals/OKRs** cascading down to tasks and up to portfolios
- **Team workload balancing** and capacity heatmaps
- **Forms → tasks** intake, approvals, and request routing
- **Recurring work, templates, and project blueprints**

3. Agile & Engineering Delivery (Jira layer)

- **Scrum/Kanban boards**, sprints, backlogs, velocity/burndown charts
- **Issue types, epics, stories, bugs** with custom fields and workflows
- **Roadmaps** synced to sprints and releases
- **Dev integrations**: Git/PR linking, CI/CD status, deployment tracking
- **Automation rules** (triggers/conditions/actions) and audit logs
- **Service management**: queues, SLAs, incident tickets

4. Knowledge, Docs & Databases (Notion layer)

- **Wiki/docs with @mentions** that link to tasks, issues, and people
- **Relational databases** (tables with rollups, relations, formulas) powering project data
- **Templates and page trees** for SOPs, meeting notes, specs
- **Two-way links**: a doc paragraph can reference a live task; a task can embed a doc
- **AI-assisted writing, summarization, and Q&A over workspace content**

5. Portfolio, Resource & Schedule Planning (MS Project layer)

- **Enterprise Gantt** with critical path, baselines, and dependencies
- **Resource leveling** and cost/budget tracking
- **Portfolio dashboards**: ROI, risk registers, scenario planning ("what-if" schedules)
- **Cross-project dependencies** and program management
- **Earned value / progress forecasting**

6. Cross-Cutting "Marriage" Capabilities (the real magic)

- **One unified object graph**: a task = issue = card = database row = Gantt bar — no duplicate entry across tools
- **Universal views**: view any dataset as board, Gantt, doc, gallery, or calendar
- **Single source of truth for people, time, and money** across creative + engineering + ops
- **Cross-functional automations**: e.g., "when design shot approved → create dev story → schedule in sprint → update portfolio forecast"
- **Unified search + AI copilot** across shots, docs, tickets, and schedules

- **Role-based dashboards**: designer sees galleries; PM sees timelines; exec sees portfolio health
- **Real-time collaboration**: comments, mentions, presence — everywhere
- **Integrations hub**: Slack, GitHub, Figma, Google/MS 365, Zapier-style connectors
- **Granular permissions** and enterprise governance (SSO, audit, compliance)
- **Mobile + offline** task capture and review

7. Reporting & Intelligence

- **Custom dashboards** mixing creative output metrics, sprint velocity, budget burn, and OKR progress
- **Predictive scheduling** (AI estimates slip risk from historical data)
- **Auto-generated status reports** for stakeholders
- **Time tracking** feeding both billing and capacity planning

In one sentence: it would let a team go from *"here's an inspiring design" → "here's the spec doc" → "here's the sprint ticket" → "here's the Gantt schedule and budget" → "here's the executive dashboard"* — **without ever leaving the same workspace or re-entering data.**

Want me to sketch a concrete end-to-end workflow (e.g., a product launch) showing how one task flows through all five layers?

yes please

End-to-End Workflow: "Product Launch" Through the Hybrid App

Let's follow **one initiative** — launching a new feature (say, a **mobile app redesign**) — as it flows through all five layers. Watch how a single object morphs and accumulates context without ever being re-created.

Stage 0 — The Spark (Dribbble layer)

Maya, a designer, posts a concept shot to the workspace gallery: *"New onboarding flow — v1."*

- The shot lives in the **asset library**, tagged `#onboarding #mobile`
- She pins **3 critique comments** to specific pixels; two teammates reply inline
- She clicks **"Promote to Initiative"** → the app auto-creates a **project** ("Onboarding Redesign") with her shot attached as the brief
- **Behind the scenes:** a Notion-style **doc page** is generated ("Onboarding Redesign — Brief") pre-filled with the shot, comments, and a blank spec template
- A **portfolio entry** is created in the MS Project layer, flagged `proposed` — visible to leadership but not yet scheduled

> *No copy-paste. The shot, comments, and brief are the same object viewed differently.*

Stage 1 — Shaping the Work (Notion layer)

Priya, the PM, opens the auto-generated brief doc and turns it into a spec:

- She writes the **PRD** in the doc, embedding a **live database** of "Requirements" (each row = a future ticket)
- She adds a **rollup column** ("Est. effort") and a **formula** ("Priority score = impact × confidence ÷ effort")
- She @mentions **Sam (eng lead)** in a paragraph → Sam gets a notification and replies in-thread
- She drags the **Requirements database** into a **board view** to sanity-check grouping

> *The doc, the database, and the board are three views of one dataset. Editing a row in the board updates the doc's table instantly.*

Stage 2 — Planning the Schedule (MS Project layer)

Priya switches the project to **Gantt view**:

- Each requirement row now has a **bar, duration, dependency, and owner**
- She sets the **critical path** (design → build → QA → launch) and a **baseline**

- The app **auto-flags a conflict**: Sam is overallocated in week 3 (he's also on "Payments v2")
- She opens **Resource Leveling** → the app proposes shifting two tasks; she accepts
- A **budget** is attached; **cost per task** rolls up to a portfolio forecast
- **Portfolio dashboard** updates: this initiative now shows projected launch date, spend, and risk score

> *Same tasks from Stage 1 — now with time, money, and dependencies. Nothing re-entered.*

Stage 3 — Executing in Sprints (Jira layer)

Sam breaks the requirements into **stories and bugs**, pulls them into a **sprint**:

- The **board** shows stories moving To Do → In Progress → Done
- He links a **GitHub PR** to a story → the card auto-shows build/CI status
- **Automation rule**: "When PR merged AND QA passed → move story to Done → update Gantt % complete → notify Priya"
- A **bug** filed mid-sprint is auto-linked back to the original **design shot** (Maya gets pinged to confirm the intended behavior)
- **Burndown** and **velocity** charts feed the portfolio's forecast

> *The story is the same object as the requirement row and the Gantt bar. Its status change ripples everywhere.*

Stage 4 — Coordinating People & Comms (Asana layer)

Throughout, the **work management** layer handles the human glue:

- **Workload view**: Priya sees Maya is at 110% next week → reassigns a critique task
- **Milestone**: "Design freeze" — auto-reminds stakeholders 3 days prior
- **Approval workflow**: legal signs off on copy via a **form → task**; approval recorded as a task comment + audit log

- **Goals/OKR**: this initiative rolls up to "Improve activation rate 15%" — progress auto-calculated from task completion
- **Recurring task**: weekly status report auto-generated from live data, posted to the project's **doc page** and Slack

> *Asana's layer is the "connective tissue" — capacity, approvals, goals — all reading the same objects.*

Stage 5 — The Executive View (Portfolio layer)

The VP opens her **portfolio dashboard**:

- Sees all initiatives as **cards with health scores** (green/amber/red) derived from schedule slip, budget burn, and blocker count
- Drills into "Onboarding Redesign" → sees the **Gantt**, the **spec doc**, the **sprint board**, and the **original design shot** — all one click apart
- Runs a **what-if**: "If we add 2 engineers, launch moves 6 days earlier?" → scenario planner updates forecast
- **AI copilot**: "Summarize risks for this initiative" → returns a 3-bullet status pulling from docs, tickets, and schedule

> *She never asks "can someone send me the latest?" — the answer is live and one object deep.*

The Same Object, Five Views — Visualized

```
text

Design Shot (Dribbble)
      | promote
      ▼
Doc + Requirements DB (Notion)
      | schedule
      ▼
Gantt Bar + Budget (MS Project)
      | execute
      ▼
Story/Bug on Board (Jira)
```

```
        | coordinate
        ▼
  Workload + Approval + OKR (Asana)
        | roll up
        ▼
  Portfolio Health Card (Exec view)
```

Every arrow is a view change, not a data migration. The task's ID, comments, attachments, and history persist end-to-end.

What Made This Possible (the "marriage" mechanics)

1. **Single object graph** — task = issue = row = bar = card
2. **Universal views** — same data rendered as gallery, doc, board, Gantt, dashboard
3. **Event-driven automations** — a status change in Jira updates Gantt %, OKR progress, and portfolio health
4. **Bidirectional links** — doc ↔ task ↔ shot ↔ PR, all navigable
5. **Role-scoped lenses** — designer sees galleries, PM sees timelines, exec sees portfolio — same underlying truth

Want me to go deeper on any one stage — e.g., **the data model** that makes one object render five ways, or a **day-in-the-life** of a single team member using it?

> Create a project design documentation of this website with all this features listed and a deep dive of all the features and the collaborations and integration needs

Project Design Documentation

Working Title: Nexus — Unified Work & Creative Operating System

Version: 1.0 (Design Spec)
Status: Concept / Pre-build
Audience: Product, Design, Engineering, Leadership

1. Executive Summary

Nexus is a unified workspace that merges the strengths of five category-defining tools into one object graph:

Layer	Inspired by	Core Contribution
Visual / Creative	Dribbble	Asset galleries, visual critique, portfolio-to-deliverable linking
Work Management	Asana	Tasks, views, goals, workload, approvals
Agile Delivery	Jira	Sprints, boards, issues, dev integration, automation
Knowledge & Data	Notion	Docs, relational databases, templates, AI
Portfolio & Schedule	MS Project	Gantt, critical path, resources, budgets, scenarios

Core thesis: A task, issue, card, database row, Gantt bar, and portfolio line are **the same object viewed differently**. No re-entry, no sync drift, one source of truth.

2. Product Vision & Principles

Vision: "From inspiration to executive dashboard — one workspace, one object, zero re-entry."

Design principles:

1. **One object, many views** — data is stored once; presentation is a lens.
2. **Progressive depth** — a solo designer can use it lightly; an enterprise can go deep without switching tools.
3. **Everything is linkable** — every object can reference every other object.
4. **Automation is first-class** — events ripple across layers.
5. **Role-scoped truth** — same data, tailored lenses.

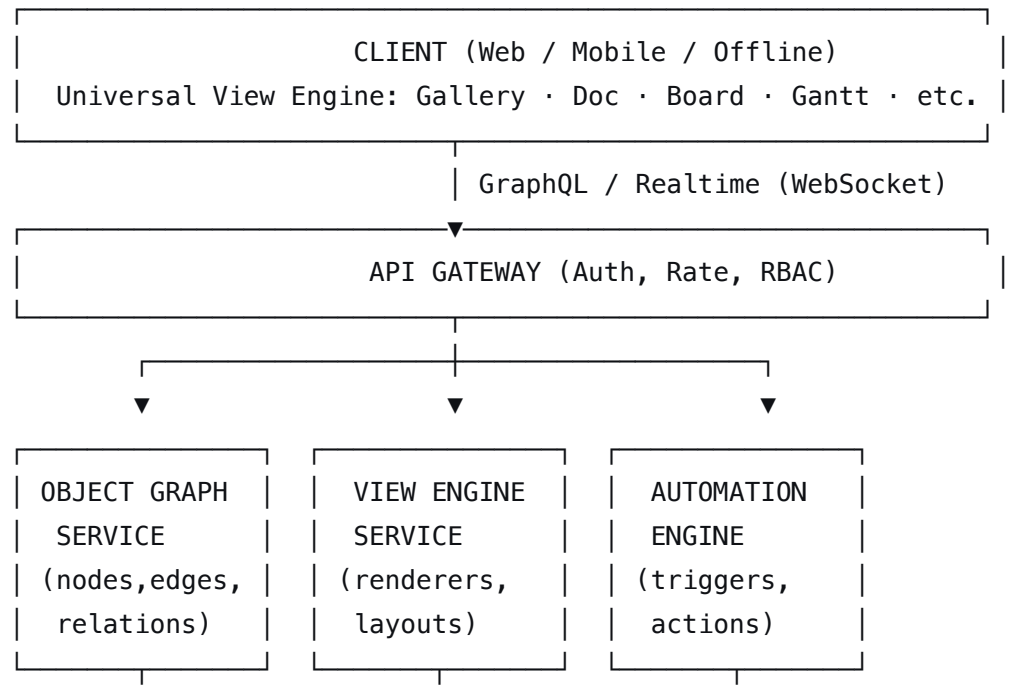
6. **No silent data loss** — history, audit, and versioning everywhere.

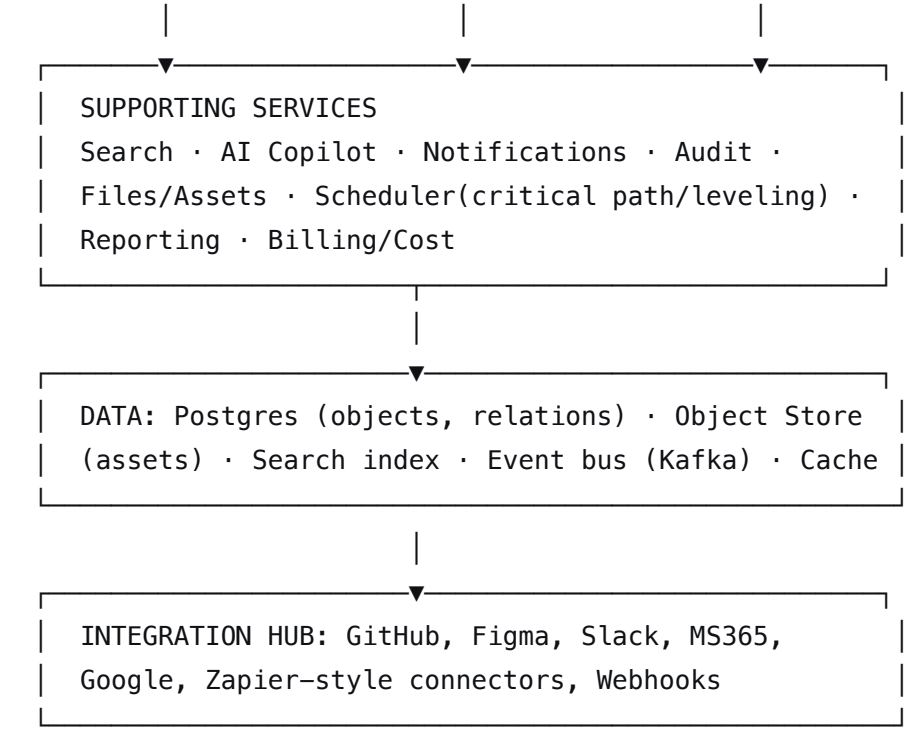
3. Target Users & Personas

Persona	Role	Primary Layers	Key Needs
Maya	Product Designer	Dribbble + Notion	Visual critique, asset versioning, brief-to-spec
Priya	Project/Program Manager	Asana + MS Project	Timeline, capacity, budget, goals
Sam	Engineering Lead	Jira + Notion	Sprints, dev integration, specs
The VP	Executive	Portfolio	Health, forecast, risk, ROI
Ops/Legal	Support functions	Asana	Approvals, intake forms, SLAs

4. System Architecture (High Level)

text





Key architectural decisions:

- **Single Object Graph Service** is the system of record. All five "layers" are *views and field-sets* over it, not separate databases.
- **View Engine** is pluggable — new views (e.g., timeline, map) added without touching data.
- **Event bus** powers automations, real-time collab, and integrations.
- **Scheduler service** handles critical path, resource leveling, and scenario what-ifs.

5. The Unified Object Model (Deep Dive)

5.1 Core Object Types

Object	Description	Inherits concepts from
Workspace	Top-level tenant	All
Space	Container (team/project area)	Asana project / Jira project
Initiative	Portfolio-level work item	MS Project project

Object	Description	Inherits concepts from
WorkItem	The universal task (task=issue=card=row=bar)	All
Doc	Rich-text page	Notion
Database	Relational table of objects	Notion
Asset	Image/file/shot with versions	Dribbble
Person	User	All
Goal	OKR	Asana
Sprint	Time-boxed batch	Jira
Milestone	Date-bound checkpoint	Asana / MS Project
Approval	Sign-off record	Asana
Comment	Threaded discussion (optionally pixel-anchored)	All
View	Saved lens over objects	All

5.2 WorkItem — The Universal Object

Every WorkItem carries a **superset of fields**, most optional and revealed by context:

```
text

WorkItem
├── identity: id, type (task|story|bug|epic|requirement), title, description
├── state: status, workflow_id, priority, resolution
├── people: assignee(s), reporter, watchers, reviewers
├── time: created, updated, due, start, duration, baseline_start/end
├── structure: parent_id, children[], dependencies[], blocks/blocked_by
├── planning: sprint_id, milestone_id, initiative_id, estimate, story_points
├── money: cost_estimate, cost_actual, budget_code
├── creative: linked_assets[], design_versions[], critique_threads[]
├── knowledge: linked_docs[], spec_section
├── dev: linked_prs[], branch, ci_status, deployment
├── custom: custom_fields{} (per workflow schema)
```

```
  ├── access: permissions, visibility
  └── history: audit_log[], activity_feed[]
```

Why this works: A designer's "shot" and an engineer's "story" and a PM's "Gantt bar" are the *same row* with different fields populated and different views applied.

5.3 Relations (the graph edges)

```
text

WorkItem ──depends_on──▶ WorkItem
WorkItem ──belongs_to──▶ Sprint / Milestone / Initiative
WorkItem ──implements──▶ Requirement (a Database row)
WorkItem ──references──▶ Doc / Asset
Initiative ──rolls_up_to──▶ Goal
Asset ──versioned_as──▶ AssetVersion
Comment ──anchored_to──▶ Asset (pixel coords) | Doc (range) | WorkItem
Person ──assigned_to──▶ WorkItem
Approval ──gates──▶ Milestone / WorkItem
```

Relations are **bidirectional and typed**, enabling rollups (e.g., "cost of all children") and reverse queries ("what docs reference this asset?").

5.4 The "Same Object, Five Views" Mechanic

View	Renders WorkItems as	Key fields surfaced
Gallery	Visual cards with asset thumbnails	creative, title, status
Doc	Inline embeds / database blocks	knowledge, custom
Board	Kanban columns by status	state, assignee
Gantt	Time bars with dependencies	time, structure, planning
Portfolio	Health cards & rollups	money, goals, risk

A change to `status` fires an event → Board moves card, Gantt recalculates %, Portfolio health updates, OKR progress rolls up. **One write, many reads.**

6. Feature Deep Dive (by Layer)

6.1 Visual & Creative Layer (Dribbble-inspired)

Features:

- **Shot galleries** — grid/masonry of assets, filterable by tag, project, person
- **Asset versioning** — v1→vN with diff view and rollback
- **Pixel-anchored critique** — comments pinned to coordinates on an image; resolve threads
- **Mood boards** — collections of assets across sources, shareable
- **Asset library** — licensing metadata, usage tracking, rights expiry alerts
- **Promote-to-initiative** — one click turns a shot into a project/brief
- **Design-system hub** — tokens, components, linked to dev tickets
- **Figma integration** — embed live frames, sync versions

Data: `Asset` , `AssetVersion` , `Comment(anchored)` , `Collection`
Collaboration needs: real-time cursor presence on galleries, threaded critique, @mentions, reaction emoji.

6.2 Work Management Layer (Asana-inspired)

Features:

- **Multi-view tasks** — list, board, timeline, calendar, workload, Gantt (shared engine)
- **Dependencies & milestones** with visual conflict flags
- **Automations** — "if status=Done → assign reviewer → notify channel"
- **Goals/OKRs** — cascade to work items; auto-progress from completion
- **Workload & capacity** — heatmaps, overallocation alerts, leveling suggestions
- **Forms → tasks** — intake with routing rules
- **Approvals** — multi-step, recorded with audit trail
- **Templates & blueprints** — reusable project structures
- **Recurring tasks** — schedule-based regeneration

Data: `WorkItem` , `Goal` , `Approval` , `Form` , `Template`
Collaboration needs: comments, mentions, approval threads, notification digests, guest access.

6.3 Agile Delivery Layer (Jira-inspired)

Features:

- **Scrum/Kanban boards** — sprints, backlogs, WIP limits
- **Issue hierarchy** — Epic → Story → Subtask; Bug type
- **Custom workflows** — states, transitions, conditions, validators
- **Sprint ceremonies** — planning, burndown, velocity, retrospective notes
- **Dev integrations** — GitHub/GitLab/Bitbucket PR linking, branch naming, CI status, deployments
- **Automation rules** — trigger/condition/action builder
- **Service management** — queues, SLAs, incident tickets
- **Audit log** — every transition recorded

Data: `WorkItem(type=story|bug|epic)` , `Sprint` , `Workflow` , `AutomationRule`
Collaboration needs: dev-channel notifications, PR review sync, sprint standup views.

6.4 Knowledge & Data Layer (Notion-inspired)

Features:

- **Docs** — rich text, embeds, @mentions linking to any object
- **Relational databases** — relations, rollups, formulas, filters, sorts, group-bys
- **Multiple database views** — table, board, gallery, calendar, timeline (reuses View Engine)
- **Templates** — page and database templates
- **Wiki/sidebar tree** — nested pages, teamspaces
- **Two-way linking** — doc paragraph ↔ live task; task embeds doc section
- **AI copilot** — summarize, Q&A over workspace, generate specs

- **Version history** — page-level diffs and restore

Data: `Doc`, `Database`, `DatabaseRow` (= WorkItem with custom fields), `Block`
Collaboration needs: real-time co-editing (CRDT/OT), inline comments, presence, permissions inheritance.

6.5 Portfolio & Schedule Layer (MS Project-inspired)

Features:

- **Enterprise Gantt** — critical path, baselines, dependencies, lag/lead
- **Resource leveling** — auto-resolve overallocation with suggestions
- **Budget & cost tracking** — per-task cost, rollup to initiative/portfolio
- **Portfolio dashboards** — health scores, ROI, risk registers
- **Scenario planning** — what-if schedules ("add 2 engineers → 6 days earlier")
- **Cross-project dependencies** — program management
- **Earned value / forecasting** — progress vs. plan
- **Risk register** — probability × impact, mitigation tracking

Data: `Initiative`, `Dependency`, `Baseline`, `Budget`, `Risk`, `Scenario`
Collaboration needs: exec dashboards (read-only lenses), scenario sharing, approval gates on baseline changes.

6.6 Cross-Cutting Capabilities (the "Marriage")

Capability	Description
Unified object graph	Single source of truth; views are lenses
Universal View Engine	Any dataset → any view (gallery/doc/board/Gantt/calendar)
Event-driven automations	Status change ripples across layers
Unified search	One query across shots, docs, tickets, schedules; semantic + keyword

Capability	Description
AI Copilot	Summarize, estimate, Q&A, generate specs from shots/comments
Role-based dashboards	Designer/PM/Eng/Exec lenses over same data
Real-time collaboration	Presence, cursors, live edits everywhere
Granular permissions	Object-level, field-level, inheritance, guest access
Audit & compliance	Immutable log, SSO, SCIM, data residency
Mobile + offline	Capture, review, approve offline; sync on reconnect
Integration hub	Connectors + webhooks + Zapier-style recipes

7. Collaboration Needs (Deep Dive)

7.1 Real-Time Collaboration

- **Presence & cursors** on docs, galleries, boards
- **Live co-editing** (CRDT-based) for docs and database cells
- **Live board updates** via WebSocket (card moves visible instantly)
- **Conflict resolution** — last-write-wins with merge UI for docs; optimistic locking for structured fields

7.2 Communication Surfaces

- **Threaded comments** on any object (doc range, image pixel, task, Gantt bar)
- **@mentions** resolving to people, objects, or docs
- **Reactions & emoji**
- **Notification engine** — in-app, email, Slack, digest; per-user rules
- **Activity feeds** per object, per project, per person

7.3 Approval & Governance

- **Multi-step approvals** with roles, deadlines, escalation
- **Audit trail** on every state change, approval, permission edit
- **Guest access** with scoped visibility
- **Field-level permissions** (e.g., budget hidden from contractors)

7.4 Cross-Functional Coordination

- **Shared milestones** visible across layers
- **Handoff automations** (design approved → dev story created)
- **Single notification truth** — no duplicate alerts across tools
- **Role-scoped dashboards** — same data, tailored lens

8. Integration Needs (Deep Dive)

8.1 Integration Categories

Category	Examples	Purpose
Design	Figma, Sketch, Adobe CC	Asset sync, live frames, version links
Code	GitHub, GitLab, Bitbucket	PR linking, CI status, deployments
Communication	Slack, MS Teams, Discord	Notifications, unfurling, commands
Productivity	Google Workspace, MS 365	Docs import, calendar sync, SSO
Automation	Zapier, Make, native webhooks	Cross-tool recipes
Identity	Okta, Azure AD, Google	SSO, SCIM provisioning
Storage	Google Drive, OneDrive, Dropbox	Asset linking
Analytics	BI tools, data warehouse export	Reporting

Category	Examples	Purpose
Time/Billing	Harvest, Toggl	Time tracking sync
Support	Zendesk, Intercom	Ticket ingestion

8.2 Integration Architecture

```
text

Integration Hub
├── Connectors (OAuth, polling, webhooks)
├── Event Mapper (external event → internal object event)
├── Field Mapper (schema translation)
├── Conflict Resolver (source-of-truth rules per field)
├── Rate Limiter & Retry Queue
└── Observability (logs, metrics, dead-letter queue)
```

Rules:

- Each integration declares **field ownership** (which side wins on conflict)
- All external events flow through the **event bus** so automations fire uniformly
- Connectors are sandboxed; failures don't block core
- **Webhook-first**, polling fallback

8.3 Key Integration Flows

- **GitHub ↔ WorkItem:** PR opened → link card; CI pass → status badge; merge → transition + notify
- **Figma ↔ Asset:** frame published → new AssetVersion; comments sync both ways
- **Slack ↔ Notifications:** mention → DM; card move → channel message; slash commands to create items
- **MS365/Google ↔ Docs/Calendar:** import docs, sync milestones to calendars
- **Identity ↔ Access:** SSO login, SCIM user/group sync, role mapping

9. Non-Functional Requirements

Area	Requirement
Performance	View render < 300ms for 10k-object datasets (virtualized)
Scalability	100k objects/workspace; 10k concurrent users/tenant
Availability	99.95% uptime; graceful degradation of non-core features
Security	SOC 2 Type II, encryption at rest/in transit, field-level RBAC
Compliance	GDPR, data residency options, audit export
Accessibility	WCAG 2.1 AA across all views
Offline	Core capture/review/approve offline; conflict-aware sync
Extensibility	Public API, webhooks, plugin SDK for custom views

10. Risks & Open Questions

Risk	Mitigation
Feature sprawl / complexity	Progressive disclosure; persona-based onboarding; "simple mode"
Performance of unified graph	Materialized views per lens; caching; query planner
Integration conflict resolution	Explicit field ownership; conflict UI
Adoption (too many tools at once)	Land-and-expand: start with one layer, add others
Data migration from 5 tools	Importers with mapping UI; dual-run period
AI hallucination in copilot	Grounded retrieval; citations to source objects

Open questions:

- Should layers be **togglable** per workspace (e.g., hide Gantt for small teams)?

- Pricing model: per-seat, per-layer, or usage-based?
- How deep should offline support go for Gantt editing?

11. Suggested Roadmap (Phased)

Phase	Scope	Outcome
P1 — Foundation	Object graph, View Engine, Docs + Tasks + Board	Usable Notion+Asana hybrid
P2 — Delivery	Sprints, dev integration, automations	Jira-replacement viable
P3 — Creative	Galleries, asset versioning, critique	Dribbble layer
P4 — Portfolio	Gantt, critical path, resources, budgets	MS Project layer
P5 — Intelligence	AI copilot, search, cross-layer automations	The "marriage" moment
P6 — Ecosystem	Integration hub, API, plugin SDK	Enterprise-ready

12. Appendices

- **A. Glossary** — WorkItem, Initiative, Lens, Object Graph
- **B. View Engine Spec** — renderer contract, layout schema
- **C. Automation DSL** — trigger/condition/action grammar
- **D. Permission Model** — inheritance rules, field-level matrix
- **E. Integration Field-Ownership Matrix** — per connector

End of Design Documentation v1.0

Would you like me to expand any section into a **full technical spec** — e.g., the **Object Graph schema (SQL/GraphQL)**, the **View Engine contract**, or the **Automation DSL**?

please proceed

Technical Specification Expansion

Nexus — Unified Work & Creative Operating System

This document expands the three highest-risk technical areas from the design doc:

- **Part A:** Object Graph Schema (data model + GraphQL contract)
- **Part B:** View Engine Contract (renderer/layout spec)
- **Part C:** Automation DSL (grammar, execution, examples)

PART A — Object Graph Schema

A.1 Storage Strategy

Hybrid polyglot persistence:

Store	Role
PostgreSQL	System of record for objects, relations, history
Redis	Cache, presence, session, rate limits
Object storage (S3-compatible)	Assets, versions, exports
Search index (OpenSearch/Elastic)	Keyword + semantic search
Event bus (Kafka)	Automations, integrations, real-time fan-out
Materialized view store	Per-lens precomputed projections (Gantt, portfolio)

Why Postgres-first: relations are typed and queryable; JSONB handles custom fields; row-level security maps cleanly to permissions; mature tooling.

A.2 Core Tables (PostgreSQL DDL — simplified)

```sql
-- ============ TENANCY ============
CREATE TABLE workspaces (
  id            UUID PRIMARY KEY,
  name          TEXT NOT NULL,
  plan          TEXT NOT NULL,          -- free|team|enterprise
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE spaces (
  id            UUID PRIMARY KEY,
  workspace_id  UUID REFERENCES workspaces(id),
  name          TEXT NOT NULL,
  kind          TEXT,                   -- team|project|portfolio
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- ============ UNIVERSAL OBJECT ============
CREATE TABLE work_items (
  id              UUID PRIMARY KEY,
  workspace_id    UUID REFERENCES workspaces(id),
  space_id        UUID REFERENCES spaces(id),
  parent_id       UUID REFERENCES work_items(id),
  type            TEXT NOT NULL,        -- task|story|bug|epic|requirement|milest
one
  title           TEXT NOT NULL,
  description     JSONB,                -- rich text blocks
  status          TEXT,
  workflow_id     UUID,
  priority        SMALLINT,
  assignee_ids    UUID[],               -- denormalized for speed
  reporter_id     UUID,
  sprint_id       UUID,
  initiative_id   UUID,
  start_at        TIMESTAMPTZ,
  due_at          TIMESTAMPTZ,
  duration_days   NUMERIC,
  baseline_start  TIMESTAMPTZ,
  baseline_end    TIMESTAMPTZ,
  estimate        NUMERIC,
```

```
  story_points    NUMERIC,
  cost_estimate   NUMERIC,
  cost_actual     NUMERIC,
  custom_fields   JSONB DEFAULT '{}',
  permissions     JSONB,                -- field-level overrides
  version         BIGINT DEFAULT 1,     -- optimistic concurrency
  created_at      TIMESTAMPTZ DEFAULT now(),
  updated_at      TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_wi_space_status ON work_items(space_id, status);
CREATE INDEX idx_wi_assignee ON work_items USING GIN(assignee_ids);
CREATE INDEX idx_wi_custom ON work_items USING GIN(custom_fields);
CREATE INDEX idx_wi_sprint ON work_items(sprint_id);

-- ============ RELATIONS (typed edges) ============
CREATE TABLE relations (
  id            UUID PRIMARY KEY,
  src_id        UUID NOT NULL,
  src_type      TEXT NOT NULL,
  dst_id        UUID NOT NULL,
  dst_type      TEXT NOT NULL,
  rel_type      TEXT NOT NULL,          -- depends_on|blocks|implements|reference
s|rolls_up_to|...
  metadata      JSONB DEFAULT '{}',
  created_at    TIMESTAMPTZ DEFAULT now(),
  UNIQUE(src_id, dst_id, rel_type)
);

CREATE INDEX idx_rel_src ON relations(src_id);
CREATE INDEX idx_rel_dst ON relations(dst_id);
CREATE INDEX idx_rel_type ON relations(rel_type);

-- ============ KNOWLEDGE ============
CREATE TABLE docs (
  id            UUID PRIMARY KEY,
  workspace_id  UUID REFERENCES workspaces(id),
  space_id      UUID REFERENCES spaces(id),
  parent_id     UUID REFERENCES docs(id),
  title         TEXT,
  blocks        JSONB NOT NULL,         -- block tree (Notion-style)
  version       BIGINT DEFAULT 1,
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE databases (
  id            UUID PRIMARY KEY,
```

```
  workspace_id  UUID REFERENCES workspaces(id),
  schema        JSONB NOT NULL,         -- field definitions, formulas, rollups
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- A database row IS a work_item; link table maps rows to DBs
CREATE TABLE database_rows (
  database_id   UUID REFERENCES databases(id),
  work_item_id  UUID REFERENCES work_items(id),
  PRIMARY KEY(database_id, work_item_id)
);

-- ============ ASSETS ============
CREATE TABLE assets (
  id            UUID PRIMARY KEY,
  workspace_id  UUID REFERENCES workspaces(id),
  title         TEXT,
  media_type    TEXT,                   -- image|video|file
  license       TEXT,
  tags          TEXT[],
  current_version_id UUID,
  created_at    TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE asset_versions (
  id            UUID PRIMARY KEY,
  asset_id      UUID REFERENCES assets(id),
  version_no    INT NOT NULL,
  storage_key   TEXT NOT NULL,
  width         INT, height INT,
  checksum      TEXT,
  created_by    UUID,
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- ============ COLLABORATION ============
CREATE TABLE comments (
  id            UUID PRIMARY KEY,
  target_id     UUID NOT NULL,
  target_type   TEXT NOT NULL,          -- work_item|doc|asset|gantt_bar
  anchor        JSONB,                  -- {x,y} pixel | {from,to} text range
  parent_id     UUID REFERENCES comments(id),
  author_id     UUID,
  body          JSONB,
  resolved      BOOLEAN DEFAULT false,
  created_at    TIMESTAMPTZ DEFAULT now()
);
```

```
CREATE TABLE approvals (
  id            UUID PRIMARY KEY,
  target_id     UUID NOT NULL,
  target_type   TEXT NOT NULL,
  steps         JSONB NOT NULL,         -- ordered approvers
  current_step  INT DEFAULT 0,
  state         TEXT,                   -- pending|approved|rejected|expired
  created_at    TIMESTAMPTZ DEFAULT now()
);

-- ============ AUDIT (append-only) ============
CREATE TABLE audit_log (
  id            BIGSERIAL PRIMARY KEY,
  workspace_id  UUID,
  actor_id      UUID,
  action        TEXT,
  target_id     UUID,
  target_type   TEXT,
  before        JSONB,
  after         JSONB,
  at            TIMESTAMPTZ DEFAULT now()
);
-- Partitioned monthly; immutable via trigger.

-- ============ PERMISSIONS ============
CREATE TABLE roles (
  id            UUID PRIMARY KEY,
  workspace_id  UUID,
  name          TEXT,
  grants        JSONB NOT NULL          -- {object_type: [actions], field_masks:
[]}
);

CREATE TABLE role_assignments (
  id            UUID PRIMARY KEY,
  principal_id  UUID NOT NULL,          -- user|group
  principal_type TEXT NOT NULL,
  role_id       UUID REFERENCES roles(id),
  scope_id      UUID,                   -- workspace|space|object
  scope_type    TEXT
);
```

A.3 GraphQL Schema (public API)

```graphql
type WorkItem {
  id: ID!
  type: WorkItemType!
  title: String!
  description: RichText
  status: String
  priority: Int
  assignees: [Person!]!
  reporter: Person
  sprint: Sprint
  initiative: Initiative
  parent: WorkItem
  children: [WorkItem!]!
  dependencies: [Relation!]!
  dependents: [Relation!]!
  startAt: DateTime
  dueAt: DateTime
  durationDays: Float
  baselineStart: DateTime
  baselineEnd: DateTime
  estimate: Float
  storyPoints: Float
  costEstimate: Float
  costActual: Float
  customFields: JSON
  assets: [Asset!]!
  docs: [Doc!]!
  pullRequests: [PullRequest!]!
  comments(first: Int, after: String): CommentConnection!
  approvals: [Approval!]!
  history(first: Int): [AuditEntry!]!
  permissions: PermissionSet!
}

enum WorkItemType { TASK STORY BUG EPIC REQUIREMENT MILESTONE }

type Relation {
  id: ID!
  relType: RelationType!
  src: WorkItem!
  dst: WorkItem!
  metadata: JSON
}

enum RelationType { DEPENDS_ON BLOCKS IMPLEMENTS REFERENCES ROLLS_UP_TO }
```

```
input WorkItemFilter {
  spaceId: ID
  type: [WorkItemType!]
  status: [String!]
  assigneeIds: [ID!]
  sprintId: ID
  dueBefore: DateTime
  custom: JSON
  search: String
}

type Query {
  workItem(id: ID!): WorkItem
  workItems(filter: WorkItemFilter!, view: ViewSpec, first: Int, after: String):
WorkItemConnection!
  initiative(id: ID!): Initiative
  portfolio(workspaceId: ID!): Portfolio
  search(query: String!, scope: SearchScope): SearchResults!
}

type Mutation {
  createWorkItem(input: CreateWorkItemInput!): WorkItem!
  updateWorkItem(id: ID!, input: UpdateWorkItemInput!, version: Int!): WorkItem!
  moveWorkItem(id: ID!, status: String!, version: Int!): WorkItem!
  linkRelation(srcId: ID!, dstId: ID!, relType: RelationType!): Relation!
  promoteAssetToInitiative(assetId: ID!, spaceId: ID!): Initiative!
  scheduleInitiative(initiativeId: ID!, algorithm: ScheduleAlgorithm): ScheduleRe
sult!
}
```

Key contract points:

- **Optimistic concurrency** via `version` on mutations (returns `CONFLICT` with current state)
- **Pagination** cursor-based everywhere
- `view: ViewSpec` on queries lets the client request a server-projected lens (see Part B)
- **Subscriptions** (WebSocket) for `workItemUpdated`, `commentAdded`, `presenceChanged`

A.4 Event Schema (Kafka)

```
json
```

```
{
  "event_id": "uuid",
  "event_type": "work_item.status_changed",
  "workspace_id": "uuid",
  "actor_id": "uuid",
  "target": { "id": "uuid", "type": "work_item" },
  "before": { "status": "In Progress" },
  "after":  { "status": "Done" },
  "occurred_at": "ISO8601",
  "trace_id": "uuid"
}
```

Event types: `work_item.*`, `comment.*`, `approval.*`, `asset.*`, `sprint.*`, `schedule.*`, `integration.*`, `presence.*`.

Consumers: automation engine, notification service, search indexer, materialized-view builder, integration hub, audit writer.

PART B — View Engine Contract

B.1 Concept

A **View** is a *declarative lens*: dataset query + layout + field mapping + interaction config. The engine renders any compatible object set through any compatible view. Views are composable, savable, and shareable.

```
text

ViewSpec = {
  dataset: Query,             // what objects
  layout: LayoutType,         // how arranged
  fields: FieldMapping,       // which fields surface
  groupBy / sort / filter,    // organization
  interactions: InteractionConfig,
  permissions: ViewPermissions
}
```

B.2 ViewSpec Schema

```
typescript
```

```
type ViewSpec = {
  id?: string;                    // saved views have ids
  name?: string;
  dataset: {
    source: 'query' | 'database' | 'space' | 'collection';
    query: WorkItemFilter;
    includeRelations?: RelationType[];   // hydrate deps for Gantt
  };
  layout: LayoutType;
  fields: {
    primary: string;              // title field
    secondary?: string[];
    badges?: string[];            // status, priority, assignee
    hidden?: string[];            // fields to mask in this view
  };
  groupBy?: { field: string; order?: 'asc'|'desc' }[];
  sort?: { field: string; direction: 'asc'|'desc' }[];
  filters?: FilterExpression[];
  interactions: {
    selectable: boolean;
    inlineEdit: string[];         // editable fields in-place
    dragDrop?: 'status'|'date'|'parent';
    contextMenu?: string[];       // action ids
  };
  permissions: {
    visibleFields: string[];
    editableFields: string[];
  };
};

enum LayoutType {
  LIST | BOARD | GALLERY | CALENDAR | TIMELINE | GANTT | TABLE | PORTFOLIO
}
```

B.3 Renderer Contract (plugin interface)

Every layout implements this interface. New layouts register with the View Engine without touching data layer.

```
typescript

interface ViewRenderer<T = unknown> {
  type: LayoutType;

  // Validate a ViewSpec is compatible with this renderer
```

```
  validate(spec: ViewSpec): ValidationResult;

  // Project raw objects into render-ready props (server or client)
  project(objects: WorkItem[], spec: ViewSpec): RenderModel<T>;

  // React component for the layout
  Component: React.FC<RendererProps<T>>;

  // Supported interactions (engine wires these)
  capabilities: {
    dragDrop: boolean;
    inlineEdit: boolean;
    virtualization: boolean;
    grouping: boolean;
    timeAxis?: boolean;
    zoomLevels?: ('hour'|'day'|'week'|'month'|'quarter')[];
  };
}

type RendererProps<T> = {
  model: RenderModel<T>;
  spec: ViewSpec;
  onEvent: (e: ViewEvent) => void;   // select, edit, drag, drop, expand
  permissions: ViewPermissions;
};
```

B.4 Layout Specifications

LIST

- Rows = work items; sections by groupBy
- Fields: primary, secondary, badges
- Interactions: inline edit, reorder (manual sort), expand to detail

BOARD

- Columns = groupBy field values (typically status)
- Cards = primary + badges + optional asset thumb
- Interactions: drag-drop changes groupBy field; WIP limits per column
- Virtualization required for >500 cards

GALLERY

- Masonry/grid of asset-forward cards
- Primary = asset thumbnail; badges overlay
- Interactions: hover preview, click → asset detail, multi-select for collections

CALENDAR

- Items placed on `due_at` (or `start_at` – `due_at` range)
- Month/week/day zoom; drag to reschedule (updates dates)
- Overlap stacking

TIMELINE

- Horizontal bars by date; no dependency arrows
- Group by epic/initiative; collapse/expand
- Drag to shift dates; resize to change duration

GANTT

- Timeline + dependency arrows + critical path highlight
- Baseline overlay (ghost bars)
- Resource swimlanes option
- Zoom: day → quarter
- Interactions: drag/resize bars, drag dependency endpoints, level resources button
- **Scheduling hooks:** mutations call `scheduleInitiative` on drop

TABLE

- Spreadsheet grid; column = field
- Inline edit any cell; row = work item
- Column config from `fields`; custom fields supported
- Formula/rollup columns evaluated server-side

PORTFOLIO

- Cards = initiatives with health score, budget burn, schedule variance
- Drill-down to underlying Gantt/board
- Read-only lens by default; scenario toggle enables what-if

B.5 Server-Side Projection (performance)

For large datasets, the engine supports **server-projected RenderModels**:

```
text

POST /view/project
{
  "spec": ViewSpec,
  "pagination": { "cursor": "...", "limit": 200 }
}
→
{
  "model": RenderModel,     // pre-shaped for the renderer
  "nextCursor": "...",
  "totalCount": 10432
}
```

Optimizations:

- Gantt: only hydrate dependencies for visible window ± buffer
- Board: group counts precomputed; cards lazily loaded per column
- Portfolio: rollups served from materialized view store, refreshed on events

B.6 Interaction Event Bus

```
typescript

type ViewEvent =
  | { kind: 'select'; ids: string[] }
  | { kind: 'edit'; id: string; field: string; value: unknown; version: number }
  | { kind: 'drag'; id: string; from: Position; to: Position }
  | { kind: 'drop'; id: string; target: DropTarget }
  | { kind: 'expand'; id: string }
  | { kind: 'create'; at: Position; defaults: Partial<WorkItem> };

// Engine maps events → mutations → optimistic update → server ack/rollback
```

B.7 View Registry (extensibility)

```typescript
registerView({
  type: 'MAP',
  validate: (spec) => spec.dataset.source === 'query' && hasGeoField(spec),
  project: projectMapModel,
  Component: MapView,
  capabilities: { dragDrop: false, inlineEdit: true, virtualization: true, groupi
ng: false }
});
```

Third-party plugins can ship new layouts via the SDK without core changes.

PART C — Automation DSL

C.1 Design Goals

- **Declarative** — rules are data, versionable, importable/exportable
- **Typed** — validated against object schema at save time
- **Event-driven** — triggered by the event bus
- **Deterministic** — same inputs → same outputs; idempotent actions
- **Auditable** — every run logged with inputs/outputs
- **Sandboxed** — no arbitrary code; only approved actions

C.2 Grammar (EBNF-style)

```ebnf
Rule        ::= "rule" Name "{" Trigger Conditions Actions "}"
Trigger     ::= "when" EventPattern ("where" Expr)?
EventPattern::= ObjectType "." EventName
Conditions  ::= "if" Expr ("and" Expr)*
Actions     ::= "then" Action ("and" Action)*

Expr        ::= Comparison | Logical | Function | Literal
Comparison  ::= Field Op Value
Op          ::= "==" | "!=" | ">" | ">=" | "<" | "<=" | "in" | "matches"
```

```
Logical     ::= Expr ("and" | "or") Expr | "not" Expr
Function    ::= Ident "(" Args? ")"
Field       ::= Ident ("." Ident)*

Action      ::= SetAction | AssignAction | TransitionAction
              | NotifyAction | CreateAction | LinkAction
              | WebhookAction | ScheduleAction
```

C.3 Object Model (JSON representation)

```
json

{
  "id": "rule_123",
  "name": "Auto-close on merge",
  "enabled": true,
  "trigger": {
    "object": "work_item",
    "event": "pull_request.merged",
    "where": { "field": "type", "op": "==", "value": "story" }
  },
  "conditions": [
    { "field": "status", "op": "==", "value": "In Review" },
    { "field": "custom_fields.qa_passed", "op": "==", "value": true }
  ],
  "actions": [
    { "type": "transition", "to": "Done" },
    { "type": "notify", "channel": "slack:#eng-releases", "template": "merged" },
    { "type": "set", "field": "custom_fields.resolved_at", "value": "{{now}}" },
    { "type": "link", "rel": "references", "target": "{{trigger.pr.work_item_i
d}}" }
  ],
  "version": 3,
  "created_by": "user_42"
}
```

C.4 Action Catalog

Action	Parameters	Effect
set	field, value (supports `{{now}}`, `{{trigger.*}}`, `{{object.*}}`)	Update a field

Action	Parameters	Effect
assign	person or role	Set assignee
transition	to (workflow state)	Move state, validates workflow
notify	channel, template, recipients	In-app/email/Slack/Teams
create	object type, template, fields	Create related object
link	rel, target	Create relation
webhook	url, method, payload template	Call external endpoint (sandboxed, rate-limited)
schedule	initiative, algorithm	Trigger reschedule (critical path/leveling)
approve	approval template	Start approval flow
wait	duration or until event	Delay subsequent actions

C.5 Expression Language

Fields resolve against the event payload + current object state. Functions:

```
text

now(), today(),
days_between(a,b),
sum(children.cost_estimate),
count(children where status != "Done"),
contains(list, item),
starts_with(s, prefix),
lower(s), upper(s),
json_path(obj, "$.foo.bar")
```

Example condition:

```
text

if days_between(today(), due_at) < 3
   and status != "Done"
```

```
    and count(children where status != "Done") > 0
```

C.6 Execution Model

```
text

Event ⟶ Rule Matcher (indexed by object+event)
       ⟶ Evaluate "where" (fast filter)
       ⟶ Evaluate conditions (full expression)
       ⟶ Action Executor
              ├── Pre-flight validation (permissions, workflow)
              ├── Optimistic apply + emit downstream events
              ├── Retry policy (exponential backoff, max 5)
              └── Dead-letter on exhaustion
       ⟶ Audit log entry (inputs, outputs, duration)
```

Guarantees:

- **At-least-once delivery** with idempotency keys per (rule, event)
- **Ordering** per object (partition key = object id)
- **Loop protection**: rules cannot trigger themselves more than N times (configurable depth limit, default 5); cycles detected and halted
- **Permissions**: rules run as a service principal but *cannot exceed* the creating user's grants

C.7 Sample Rules (copy-paste ready)

1. Design-approved → create dev story

```
json

{
  "name": "Design handoff",
  "trigger": { "object": "approval", "event": "approved",
               "where": { "field": "target_type", "op": "==", "value": "asset" }
},
  "conditions": [ { "field": "target.tags", "op": "in", "value": "handoff" } ],
  "actions": [
    { "type": "create", "object": "work_item",
      "template": "dev_story",
      "fields": { "title": "Implement: {{target.title}}",
                  "type": "story",
```

```
                "initiative_id": "{{target.initiative_id}}",
                "custom_fields.spec_asset_id": "{{target.id}}" } },
    { "type": "notify", "channel": "slack:#eng", "template": "new_story" }
  ]
}
```

2. Sprint rollover

```
json

{
  "name": "Sprint rollover",
  "trigger": { "object": "sprint", "event": "closed" },
  "actions": [
    { "type": "set", "field": "sprint_id", "value": "{{next_sprint.id}}",
      "where": { "field": "status", "op": "!=", "value": "Done" } },
    { "type": "notify", "channel": "#standup", "template": "rolled_over" }
  ]
}
```

3. Budget overrun alert

```
json

{
  "name": "Budget guard",
  "trigger": { "object": "initiative", "event": "cost_updated" },
  "conditions": [
    { "field": "cost_actual", "op": ">", "value": "{{budget * 0.9}}" }
  ],
  "actions": [
    { "type": "notify", "channel": "email:finance@", "template": "budget_alert"
},
    { "type": "set", "field": "custom_fields.health", "value": "amber" }
  ]
}
```

4. Critical-path slip → escalate

```
json

{
  "name": "Slip escalation",
  "trigger": { "object": "work_item", "event": "dates_changed" },
  "conditions": [
```

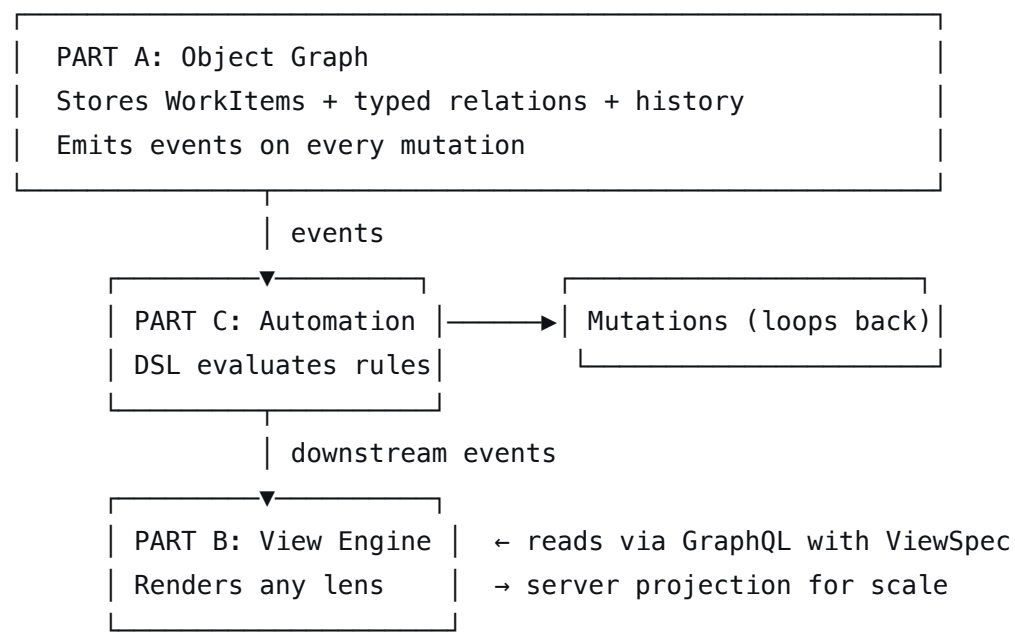
```
      { "field": "on_critical_path", "op": "==", "value": true },
      { "field": "due_at", "op": ">", "value": "{{baseline_end}}" }
    ],
    "actions": [
      { "type": "assign", "person": "{{initiative.owner}}" },
      { "type": "notify", "channel": "slack:#portfolio", "template": "slip_alert"
},
      { "type": "approve", "template": "schedule_change" }
    ]
}
```

C.8 DSL Versioning & Testing

- Rules carry `version`; changes create new versions with diff history
- **Dry-run mode**: evaluate against historical events, show what *would* fire
- **Test harness**: fixtures of events + expected actions; runs in CI
- **Import/export** as JSON or YAML
- **Marketplace**: share rule packs (e.g., "SOC2 approval flow")

Summary — How the Three Parts Interlock

text



- **A** is the truth; **B** is how humans see it; **C** is how it acts on itself.

- All three share the **event bus** as the coordination spine.
- All three are **schema-driven**, so adding a field, a view, or an action is a config change — not a rewrite.

Would you like me to continue with:

- **Part D: Real-time collaboration architecture** (CRDT choice, presence, conflict UI)
- **Part E: Integration connector spec** (field-ownership matrix, OAuth, webhook contracts)
- **Part F: Permission & RBAC matrix** (full field-level model)
- Or a **developer onboarding guide** tying A/B/C into a first-week implementation plan?

> please proceed, also mention that when the user logs in they can see all their tasks in the dashboard

Technical Specification Expansion — Part D, E, F + Dashboard Home

This document continues from Parts A–C and adds:

- **Part D:** Real-time collaboration architecture
- **Part E:** Integration connector spec (field-ownership matrix, OAuth, webhook contracts)
- **Part F:** Permission & RBAC matrix (field-level model)
- **Part G:** The **Dashboard Home** — the first screen every user sees after login (their personal "all my tasks" hub)
- **Part H:** Developer onboarding guide (first-week implementation plan)

PART D — Real-Time Collaboration Architecture

D.1 Scope

Real-time collaboration must cover:

Surface	Collaboration type
Docs	Concurrent rich-text editing (multi-cursor)
Database cells	Concurrent cell edits
Boards	Live card moves, presence on cards
Gantt	Live bar drags, cursor labels
Galleries	Presence cursors, live critique threads
Comments	Threaded, live append
Presence	Who's viewing what, cursors, selections
Notifications	Fan-out to subscribed clients

D.2 Transport & Topology

```
text

Client ──WebSocket──▶ Realtime Gateway (sticky sessions)
                              │
                              ├─▶ Presence Service (Redis)
                              ├─▶ Doc Sync Service (CRDT)
                              ├─▶ Object Event Fan-out (Kafka consumer)
                              └─▶ Notification Service
```

- **WebSocket** for all real-time (no long-polling fallback except degraded mode)
- **Sticky routing** by `workspace_id + object_id` to a gateway pod
- **Redis pub/sub** for cross-pod presence and fan-out
- **Kafka** for durable event replay (reconnect catch-up)

D.3 CRDT Strategy

Choice: Yjs (mature, rich ecosystem, good performance).

Surface	CRDT approach
Docs (rich text)	Yjs `Y.Doc` with `Y.XmlFragment` per block tree; awareness protocol for cursors
Database cells	Per-cell `Y.Text` for text; LWW-register for scalars (with version guard)
Board status	Not CRDT — optimistic mutation with server ordering (last-write-wins + version check)
Gantt dates	Optimistic mutation with version; conflict UI if two users drag same bar
Comments	Append-only; CRDT not needed (server-ordered)

Why hybrid: CRDT for *character-level* concurrent editing; optimistic-concurrency for *structured field* changes (simpler, auditable, matches workflow semantics).

D.4 Doc Sync Protocol

```typescript
// Client connects with the doc's last known state vector
type DocSyncInit = {
  docId: string;
  stateVector: Uint8Array;   // Yjs state vector
  clientId: string;
};

// Server replies with diff since state vector
type DocSyncResponse = {
  update: Uint8Array;        // Yjs update
  serverStateVector: Uint8Array;
  presence: PresenceState[];
};

// Ongoing updates
type DocUpdate = { docId: string; update: Uint8Array; clientId: string };
```

- **Persistence:** Yjs updates are periodically compacted into a snapshot stored in `docs.blocks` (JSONB) every N updates or T seconds

- **Undo/redo:** per-user undo stack (Yjs UndoManager scoped to client)
- **History:** snapshots stored as versions; restore creates new snapshot
- **Offline:** client queues updates locally (IndexedDB); replays on reconnect; server merges via CRDT

D.5 Presence Model

```typescript
type PresenceState = {
  userId: string;
  objectId: string;          // doc, board, gantt, gallery
  cursor?: { x: number; y: number } | { from: number; to: number };
  selection?: string[];      // selected object ids
  view?: string;             // current view/lens
  color: string;             // deterministic from userId
  lastSeen: number;
};
```

- **Throttled** to 20 updates/sec per client
- **Ephemeral** — stored in Redis with TTL (60s), never persisted
- **Privacy** — user can go "invisible"; admins can disable presence per workspace

D.6 Conflict Resolution UX

Case	Behavior
Doc text	CRDT merges seamlessly; no UI needed
Same cell edit	Last-write-wins with toast: "Priya changed this field — [view]"
Gantt bar drag conflict	Show ghost of other user's position; require explicit "apply" if overlap
Status transition conflict	Workflow validator rejects invalid transitions; UI shows allowed states
Delete vs edit	Delete wins; edit shows "restored from history" option

D.7 Notification Fan-out

```typescript
// Subscriptions declared by client on connect
type Subscription = {
  scope: 'object' | 'space' | 'workspace' | 'person';
  id: string;
  events: string[];          // e.g., ["work_item.updated", "comment.added"]
};

// Server pushes only matching events to the socket
type PushEvent = {
  event: string;
  target: { id: string; type: string };
  payload: unknown;
  actor: { id: string; name: string };
  at: string;
};
```

- **Backpressure:** per-socket buffer cap (e.g., 1000 events); client acks batched
- **Reconnect:** client sends `lastEventId`; server replays from Kafka
- **Throttling:** high-frequency events (presence, cursor) are not persisted, only live

D.8 Scale & Degradation

- **Sharding:** by `workspace_id`; a workspace lives on one gateway shard
- **Fan-out limit:** documents cap concurrent editors at 50 (beyond → read-only + queue)
- **Degraded mode:** if WebSocket unavailable, fall back to polling every 15s (read-only + explicit save)

PART E — Integration Connector Specification

E.1 Connector Contract

Every integration implements this interface:

```typescript
```

```ts
interface Connector {
  id: string;                          // "github", "figma", "slack"
  displayName: string;
  authType: 'oauth2' | 'api_key' | 'webhook';
  scopes: string[];                    // requested OAuth scopes
  fieldOwnership: FieldOwnership;      // who wins on conflict
  setup(config: ConnectorConfig): Promise<ConnectorInstance>;
}

interface ConnectorInstance {
  // Inbound: external event → internal event
  handleWebhook?(req: WebhookRequest): Promise<InternalEvent[]>;
  // Outbound: internal event → external action
  handleEvent?(event: InternalEvent): Promise<void>;
  // Polling fallback
  poll?(): Promise<InternalEvent[]>;
  // Health & observability
  healthCheck(): Promise<HealthStatus>;
}

type FieldOwnership = {
  // Which side is source of truth per field
  [field: string]: 'internal' | 'external' | 'merge';
};
```

E.2 Field-Ownership Matrix (key connectors)

Field	GitHub	Figma	Slack	MS 365	Goog
WorkItem.title	—	—	—	—	—
WorkItem.status	derived from PR	—	—	—	—
WorkItem.assignee	PR author (signal)	—	—	—	—
WorkItem.due_at	—	—	—	calendar event	caler even

Field	GitHub	Figma	Slack	MS 365	Goog
Asset.current_version	—	frame version	—	—	—
Doc.body	—	—	—	imported doc	impo
Comment.body	PR comment	Figma comment	Slack thread	—	—
Person.email	—	—	Slack profile	Azure AD	Goog
Sprint.dates	—	—	—	—	—
Budget.cost_actual	—	—	—	—	—

Rules:

- `internal` = Nexus wins; external changes are logged but not applied
- `external` = source system wins; Nexus mirrors
- `merge` = both can change; conflict UI or policy (e.g., calendar wins)
- Ownership is **configurable per workspace** in enterprise plans

E.3 OAuth Flow (standard)

```
text

1. User clicks "Connect GitHub" in Integration Hub
2. Redirect → GitHub OAuth authorize (state = signed nonce)
3. Callback → exchange code for tokens
4. Store encrypted tokens (KMS) in connector_credentials table
5. Register webhooks (repo push, PR, issue events)
6. Backfill: fetch recent items for mapping
```

```
7. Field-ownership + mapping UI shown to admin
8. Connector marked "active"
```

Token storage:

```sql
CREATE TABLE connector_credentials (
  id UUID PRIMARY KEY,
  workspace_id UUID,
  connector_id TEXT,
  encrypted_tokens BYTEA,     -- KMS-encrypted
  scopes TEXT[],
  expires_at TIMESTAMPTZ,
  refresh_token_encrypted BYTEA,
  created_at TIMESTAMPTZ DEFAULT now()
);
```

Refresh: background job refreshes tokens 10 min before expiry; failures trigger admin alert.

E.4 Webhook Contracts (inbound)

GitHub → Nexus

```
POST /integrations/github/webhook
Headers: X-Hub-Signature-256, X-GitHub-Event
Body: {
  "action": "closed",
  "pull_request": {
    "number": 142,
    "merged": true,
    "head": { "ref": "feature/NEX-123-onboarding" },
    "body": "Closes NEX-123"
  },
  "repository": { "full_name": "acme/app" }
}
→ Internal events:
  - work_item.pull_request.merged (work_item_id from branch/body match)
  - work_item.status_changed (if rule applies)
```

Figma → Nexus

```
json

POST /integrations/figma/webhook
Body: {
  "event_type": "FILE_VERSION_UPDATE",
  "file_key": "abc123",
  "version_id": "v42",
  "timestamp": "2026-..."
}
→ Internal events:
  - asset.version_created (asset_id from file_key mapping)
```

Slack → Nexus

```
json

POST /integrations/slack/events
Body: {
  "type": "event_callback",
  "event": {
    "type": "message",
    "channel": "C123",
    "text": "NEX-123 needs review @priya",
    "thread_ts": "..."
  }
}
→ Internal events:
  - comment.added (linked to work_item NEX-123)
  - notification.created (mention)
```

E.5 Outbound Contracts (internal → external)

```
typescript

// Example: WorkItem status change → Slack notification
type OutboundRule = {
  connector: 'slack';
  trigger: 'work_item.status_changed';
  filter: { field: 'status', op: '==', value: 'Done' };
  action: {
    type: 'post_message';
    channel: '{{space.slack_channel}}';
```

```
      template: 'card_moved_done';
    };
  };
```

Delivery guarantees:

- At-least-once via Kafka consumer + retry queue
- Idempotency key = `event_id + connector_id`
- Dead-letter queue after 5 retries; admin dashboard shows failures
- Rate limits respected per connector (configurable token bucket)

E.6 Connector Catalog (v1 scope)

Connector	Direction	Auth	Events
GitHub	both	OAuth App	PR, push, issue, release
GitLab	both	OAuth	MR, push, issue
Figma	both	OAuth	file version, comment
Slack	both	OAuth (bot)	message, mention, command
MS Teams	both	OAuth (bot)	message, mention
Google Workspace	inbound	OAuth	doc import, calendar
MS 365	inbound	OAuth	doc import, calendar
Okta / Azure AD	inbound	SAML/OIDC + SCIM	user/group provisioning
Zapier	outbound	API key	generic actions
Webhooks (generic)	outbound	HMAC	any event

E.7 Integration Observability

- **Per-connector dashboard:** events in/out, error rate, latency, DLQ depth

- **Trace IDs** propagate from inbound webhook → internal event → outbound action
- **Replay:** admin can replay events from DLQ or by time range
- **Alerts:** error rate > 1% for 5 min → page on-call

PART F — Permission & RBAC Matrix

F.1 Model

Three-layer permission model:

```
text

1. ROLE-BASED (coarse)     — what role does the user have?
2. SCOPE-BASED (medium)    — on which objects/spaces?
3. FIELD-LEVEL (fine)      — which fields are visible/editable?
```

Precedence: field-level > scope > role (most specific wins; explicit deny always wins).

F.2 Roles (default set)

Role	Scope	Grants
Owner	Workspace	Full admin, billing, delete workspace
Admin	Workspace	Manage members, roles, integrations, settings
Space Admin	Space	Manage space members, templates, automations
Editor	Space	Create/edit work items, docs, assets
Commenter	Space	Comment, react, view
Viewer	Space	Read-only
Guest	Object (explicit)	Scoped read (+ optional comment)
Service	Workspace	API-only, scoped grants

F.3 Permission Matrix (object × action)

Action	Owner	Admin	Space Admin	Editor	Comm
View work item	✅	✅	✅	✅	✅
Create work item	✅	✅	✅	✅	❌
Edit work item	✅	✅	✅	✅	❌
Delete work item	✅	✅	✅	⚠️ own	❌
Move status	✅	✅	✅	✅	❌
Comment	✅	✅	✅	✅	✅
Approve	✅	✅	✅	⚠️ assigned	❌
Manage sprints	✅	✅	✅	✅	❌
Edit Gantt/baseline	✅	✅	✅	⚠️ assigned	❌
Manage automations	✅	✅	✅	❌	❌
Manage integrations	✅	✅	❌	❌	❌
Manage members	✅	✅	⚠️ space	❌	❌
View budget	✅	✅	✅	⚠️ config	❌
Export data	✅	✅	⚠️ space	❌	❌

✅ allowed · ❌ denied · ⚠️ conditional (own/assigned/config)

F.4 Field-Level Permissions

```
json

{
  "role": "editor",
  "field_masks": {
    "cost_estimate": "hidden",
    "cost_actual": "hidden",
    "custom_fields.salary_band": "hidden",
    "assignee_ids": "editable",
    "due_at": "editable"
  }
}
```

Enforcement:

- **GraphQL layer:** field resolvers check `permissions.visibleFields`; hidden fields return `null` + `__typename: "Redacted"`
- **API layer:** mutations validate `editableFields`; rejected with `FORBIDDEN_FIELD`
- **UI layer:** renderers hide masked fields entirely (not just disable)
- **Export layer:** masked fields excluded from CSV/JSON exports
- **Audit:** field-level access logged for sensitive fields (budget, salary)

F.5 Inheritance & Overrides

```
text

Workspace role → Space role → Object permission → Field mask
     (broad)                                      (narrow)

Rules:
- More specific scope overrides broader (object > space > workspace)
- Explicit deny always wins (even if a broader grant exists)
- Guest access is object-scoped and never inherits from space
```

Example:

- Priya is `Editor` in Space "Mobile"
- A specific initiative has `Editor` overridden to `Viewer` for Priya
- Result: Priya can edit all Mobile work items *except* that initiative

- Budget field masked for all `Editor` roles → Priya sees no budget anywhere

F.6 Service Principals (API/automation)

```json
{
  "principal_type": "service",
  "name": "github-bot",
  "grants": {
    "work_item": ["read", "transition"],
    "comment": ["create"],
    "field_masks": { "cost_actual": "hidden" }
  },
  "scope": "workspace"
}
```

Rules:

- Service principals **cannot exceed** the creating user's grants (no privilege escalation)
- Automations run as service principal but inherit the *rule creator's* field visibility
- All service actions logged with `actor_type: "service"`

F.7 Audit & Compliance

- **Every permission change** logged (who, what, when, before/after)
- **Sensitive field access** logged (budget, salary, PII fields)
- **Export events** logged with row counts and field sets
- **Retention:** configurable (default 1 year; enterprise 7 years)
- **SOC 2 / GDPR:** data subject access requests served from audit log + object graph

PART G — Dashboard Home (Post-Login "All My Tasks" Hub)

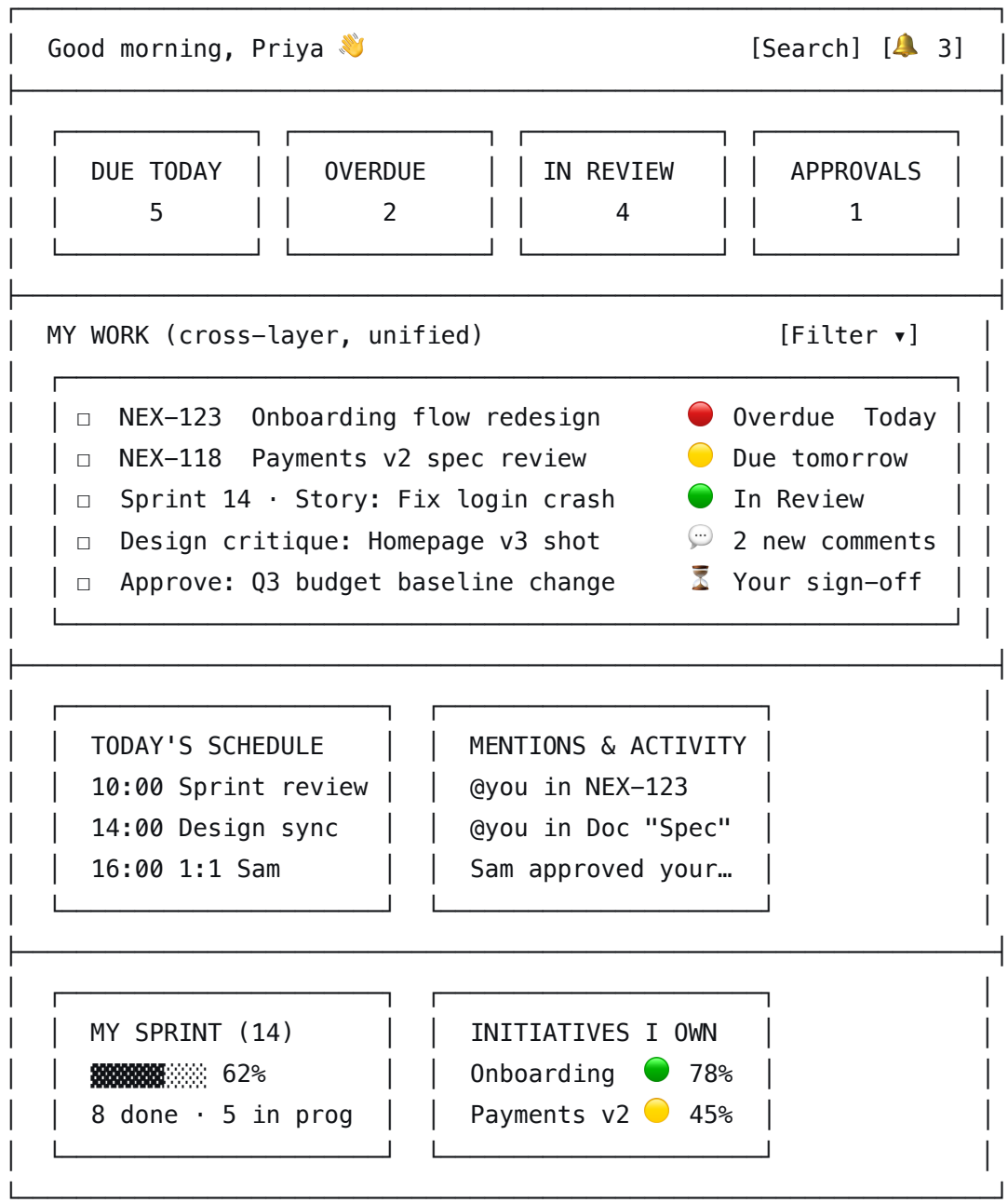
G.1 Purpose

The first screen after login is a **personal, aggregated command center** showing everything assigned to, mentioned in, watching, or needing action from the user — across *all* spaces, initiatives, sprints, and layers. This is where the "marriage" of five tools becomes immediately tangible.

> *"When the user logs in, they see all their tasks — pulled from every layer — in one prioritized dashboard, without opening a single project."*

G.2 Dashboard Composition

text



G.3 Data Sources (the "marriage" in action)

The dashboard aggregates from **every layer** via a single query:

Widget	Source objects	Layers
Stat cards	WorkItems where `assignee = me`, filtered by status/date	Asana + Jira
My Work	WorkItems (task/story/bug/requirement) + Assets (critique) + Approvals	All
Today's Schedule	Milestones, sprint ceremonies, calendar events	Asana + MS Project + calendar
Mentions & Activity	Comments, approvals, audit feed filtered to me	All
My Sprint	Sprint progress from Jira layer	Jira
Initiatives I Own	Portfolio health from MS Project layer	MS Project

Single backend query (conceptual):

```graphql
query DashboardHome {
  me {
    stats: workItemStats(assignee: ME) {
      dueToday overdue inReview pendingApprovals
    }
    myWork: workItems(
      filter: { OR: [
        { assigneeIds: [ME] },
        { watcherIds: [ME] },
        { approvalPending: ME },
        { mentions: ME }
      ]}
      view: { layout: LIST, sort: [{field: "priority"}, {field: "due_at"}] }
    ) { ...WorkItemFields }
    todaySchedule: calendarEvents(date: TODAY) { ... }
    activity: activityFeed(mentions: ME, limit: 20) { ... }
```

```
      sprint: currentSprint(member: ME) { ... }
      initiatives: initiatives(owner: ME) { ... }
    }
  }
```

G.4 Personalization

- **Widget library:** user can add/remove/reorder widgets (drag-drop)
- **Saved filters:** "My Work" defaults to assigned+watched; user can pin custom lenses
- **Role-adaptive defaults:** Designer sees gallery/critique widget first; Eng sees sprint board; Exec sees portfolio
- **Smart prioritization:** overdue > due today > mentions > in review > watching (configurable)
- **AI summary card:** "You have 5 items due today, 2 are on the critical path. Sam is blocked on your review of NEX-118." (grounded, cited)

G.5 Notifications Panel

- Bell icon shows unread count
- Panel groups by: **Assigned to me**, **Mentions**, **Approvals**, **Watching**, **System**
- Each item links directly to the object (deep link)
- **Mark all read**, **snooze**, **mute thread**

G.6 Quick Actions (from dashboard)

Action	Result
+ New task	Inline create; defaults to user's primary space
Quick status change	Inline dropdown on card (respects workflow)
Snooze	Hide until date/time
Delegate	Reassign with note
Open in view	Jump to Board/Gantt/Doc lens for that item

G.7 Performance Requirements

- **Time to first meaningful paint:** < 1.5s (cached shell + skeleton widgets)
- **Widget lazy-load:** stats + My Work first; schedule/activity/initiatives stream in
- **Pagination:** My Work shows 20, infinite scroll
- **Caching:** per-user dashboard cache invalidated on relevant events (assigned, mentioned, status change)
- **Offline:** last-known dashboard cached; banner "showing cached data"

G.8 Mobile Dashboard

- **Simplified:** stat cards + My Work list + notifications
- **Swipe actions:** complete, snooze, open
- **Offline capture:** add task/comment queued for sync

G.9 Privacy & Permissions

- Dashboard only surfaces objects the user **can already see** (permission-filtered at query time)
- Field masks respected (no budget in dashboard if masked)
- Guest users see a scoped dashboard (only their granted objects)
- **No cross-tenant leakage:** all queries scoped to `workspace_id`

PART H — Developer Onboarding Guide (First-Week Plan)

H.1 Day 1 — Environment & Orientation

- Clone monorepo; run `make dev` (Docker Compose: Postgres, Redis, Kafka, OpenSearch, MinIO)
- Seed script creates demo workspace with sample WorkItems, docs, assets, sprint, initiative
- Read: Design Doc v1, Parts A–C
- Goal: app running locally, can create a WorkItem and see it in Board view

H.2 Day 2 — Object Graph (Part A)

- Explore `work_items` table and GraphQL schema
- Add a new field end-to-end: migration → GraphQL type → resolver → UI display
- Goal: custom field visible in List view

H.3 Day 3 — View Engine (Part B)

- Study `ViewRenderer` interface
- Build a minimal new layout (e.g., "COMPACT_LIST")
- Goal: renderer registered and selectable in view switcher

H.4 Day 4 — Automations (Part C)

- Write a rule in the DSL; test dry-run mode
- Trace an event from mutation → Kafka → automation executor → downstream mutation
- Goal: "auto-assign reviewer on status change" rule working

H.5 Day 5 — Real-time & Dashboard (Parts D, G)

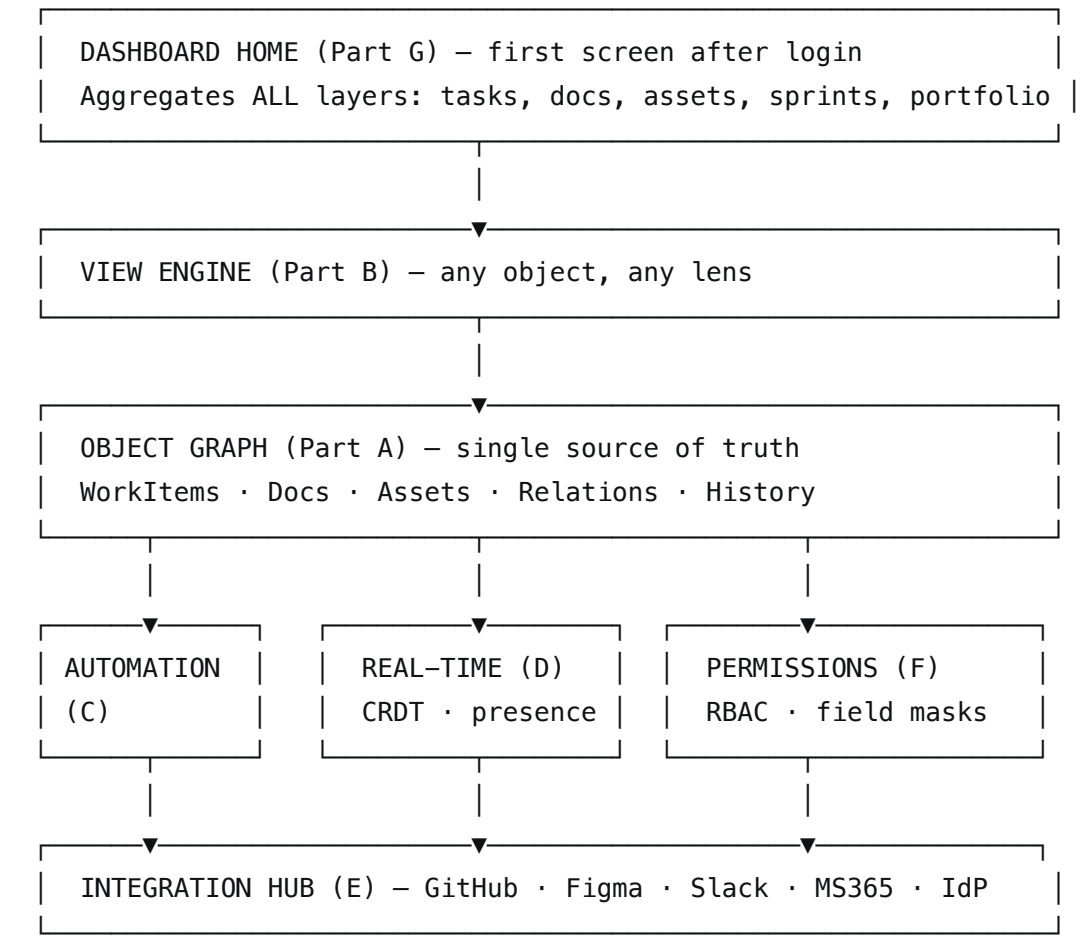
- Connect to WebSocket; observe presence on a doc
- Add a widget to the Dashboard Home (e.g., "My Overdue")
- Goal: dashboard widget live-updates when a task is completed in another tab

H.6 Week 2 Preview

- Part E (integrations): connect GitHub, see PR linked to a WorkItem
- Part F (permissions): create a role, mask a field, verify enforcement
- Ship first PR to staging

Summary — Full System Map

```text
```



The dashboard is the proof of the "marriage": a user logs in and sees — in one place — their Jira story, their Asana task, their Dribbble critique, their Notion spec review, and their MS Project approval, all permission-filtered, all live, all linkable, with zero context-switching.

Would you like me to continue with:

- **Part I: AI Copilot architecture** (grounding, retrieval, citation, guardrails)
- **Part J: Search architecture** (keyword + semantic + cross-object ranking)
- **Part K: Migration/importers** from the five legacy tools
- **Part L: Full API reference** (REST + GraphQL + webhooks)
- Or a **visual wireframe set** for the Dashboard and key views?

yes

Technical Specification Expansion — Parts I–L + Wireframes

Continuing from Parts A–H. This document covers:

- **Part I:** AI Copilot architecture (grounding, retrieval, citation, guardrails)
- **Part J:** Search architecture (keyword + semantic + cross-object ranking)
- **Part K:** Migration & importers from the five legacy tools
- **Part L:** Full API reference (REST + GraphQL + webhooks)
- **Part M:** Visual wireframes (Dashboard, Board, Gantt, Gallery, Doc)

PART I — AI Copilot Architecture

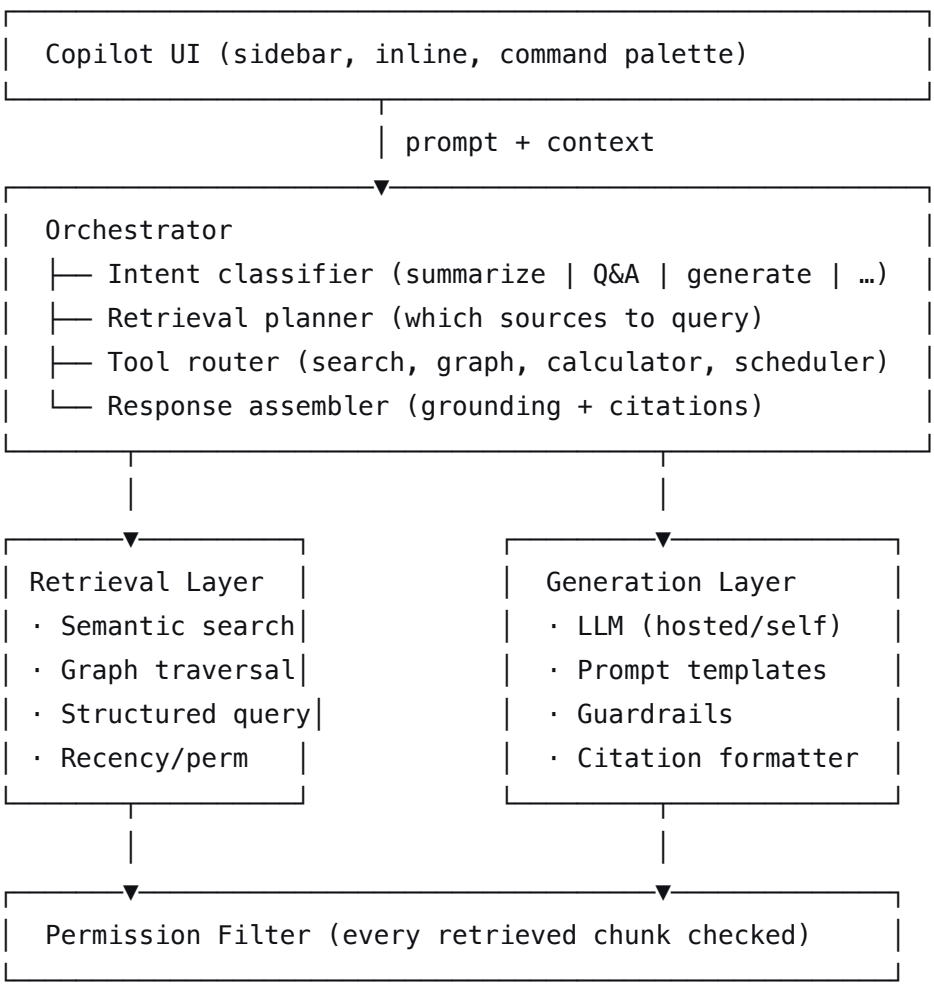
I.1 Scope & Capabilities

Capability	Example
Summarize	"Summarize this initiative's status for the exec meeting"
Q&A over workspace	"What did we decide about the onboarding flow in last week's design review?"
Generate	"Draft a spec for this shot" / "Write acceptance criteria for NEX-123"
Estimate	"Estimate effort for these 12 stories based on similar past work"
Detect risk	"Which initiatives are likely to slip?"
Find	"Show me all bugs linked to the payments redesign"
Draft comms	"Write a status update for #eng-releases"
Critique assist	"Suggest 3 critique points for this design shot" (designer-facing)

Non-goals: autonomous actions without confirmation; cross-tenant data; ungrounded generation.

I.2 Architecture

text



I.3 Retrieval Pipeline

Step 1 — Intent classification

- Small classifier model (or rule-based) maps prompt → `{intent, entities, scope}`
- Scope = workspace / space / initiative / object (from UI context)

Step 2 — Retrieval plan

text

```
Q: "What did we decide about onboarding?"
→ plan: [
```

```
    semantic_search("onboarding decisions", scope=workspace, top_k=20),
    graph_traverse(initiative="Onboarding Redesign", depth=2),
    recent_docs(tags=["decision","review"], since=30d)
  ]
```

Step 3 — Hybrid retrieval

- **Semantic:** embeddings over chunked docs, comments, specs, descriptions (chunk size ~500 tokens, overlap 50)
- **Keyword:** BM25 over titles, tags, custom fields
- **Graph:** expand from seed objects via relations (parent, children, references)
- **Structured:** direct SQL for "my tasks due this week" style queries
- **Fusion:** reciprocal rank fusion (RRF) across sources

Step 4 — Permission filter

- Every retrieved chunk carries `object_id`; filter by `can_view(user, object_id)` **before** sending to LLM
- Field-masked content excluded (e.g., budget hidden)
- Audit log records retrieval scope for sensitive queries

Step 5 — Rerank

- Cross-encoder reranker (small model) on top 50 → top 8 chunks
- Recency boost for "recent decisions" intents

I.4 Grounding & Citations

Every generated claim that references workspace content must include an inline citation:

```
text

The team decided to postpone payments v2 to Q3 [1] and
prioritize onboarding instead [2].

[1] Doc "Q2 Planning Notes" · updated 2026-03-12 · §Decisions
[2] Comment on NEX-118 by Sam · 2026-03-14
```

Citation contract:

```typescript
type Citation = {
  objectId: string;
  objectType: 'doc'|'work_item'|'asset'|'comment';
  chunkId: string;
  snippet: string;          // ≤200 chars
  deepLink: string;         // opens object at anchor
};
```

Rule: if no citation can be produced, the response is flagged `ungrounded: true` and displayed with a warning banner.

I.5 Tool Use (function calling)

The copilot can call internal tools with user confirmation:

Tool	Description	Confirmation
`search`	Hybrid search	No
`get_object`	Fetch by id	No
`list_work_items`	Filtered query	No
`summarize_thread`	Comment thread → summary	No
`estimate_effort`	Historical estimate model	No
`create_work_item`	Creates object	**Yes** (preview)
`update_status`	Transitions	**Yes**
`schedule_scenario`	Runs what-if	No (read-only)
`draft_doc`	Creates doc draft	**Yes**

Confirmation UI: "I'll create a story 'Implement onboarding v2' in Sprint 14. [Confirm] [Edit] [Cancel]"

I.6 Guardrails

Guardrail	Implementation
Tenant isolation	Retrieval scoped to `workspace_id`; embeddings partitioned
Permission respect	Filter before LLM; never leak masked fields
Prompt injection	User content sanitized; system prompt delimited; tool calls validated
Hallucination	Citation required for factual claims; `ungrounded` flag
PII	Redaction pass on retrieved chunks (regex + NER)
Rate limits	Per-user token budget; per-workspace spend cap
Audit	Every prompt, retrieval set, and response logged (redacted)
Opt-out	Workspace admin can disable copilot; user can disable personal history

I.7 Model Strategy

- **Default:** hosted frontier model via provider abstraction (swap-friendly)
- **Self-hosted option** for enterprise (data residency)
- **Small models** for classification, reranking, embedding (cost control)
- **Embedding model:** consistent per workspace; re-index on change
- **Caching:** identical prompt + context hash → cached response (TTL 1h)

I.8 Evaluation

- **Golden set:** 200 Q&A pairs with expected citations (per release)
- **Metrics:** citation accuracy, groundedness, answer relevance, latency p95
- **Human review:** weekly sample of 50 responses by PM/eng
- **Regression gate:** block release if citation accuracy drops >2%

PART J — Search Architecture

J.1 Search Types

Type	Use case	Example
Quick find (⌘K)	Navigate to known object	"NEX-123", "Priya", "onboarding doc"
Full search	Explore workspace	"bugs in payments linked to redesign"
Scoped search	Within a space/initiative	filter chips in UI
Semantic	Natural-language questions	"decisions about onboarding"
Filter search	Structured query builder	status=Done AND assignee=me

J.2 Index Design

OpenSearch indices (per workspace shard):

```
json

// index: wi_{workspace_id}
{
  "id": "NEX-123",
  "type": "work_item",
  "title": "Onboarding flow redesign",
  "description_text": "...",
  "status": "In Progress",
  "assignees": ["priya"],
  "tags": ["onboarding", "mobile"],
  "space": "mobile",
  "initiative": "onboarding-redesign",
  "sprint": "sprint-14",
  "custom_text": "...",           // searchable custom fields
  "embedding": [0.12, -0.03, ...],// 1536-dim vector
  "updated_at": "2026-...",
  "acl": ["workspace", "space:mobile"]  // permission filter
}
```

Separate indices: `docs`, `assets`, `comments`, `people` — unified via cross-index query.

J.3 Ranking (hybrid)

```
text

final_score = α · BM25 + β · cosine_similarity + γ · recency + δ · authority

α=0.35  β=0.45  γ=0.10  δ=0.10
```

- **Authority:** object's relation centrality (linked by many), owner status, pinned
- **Recency:** exponential decay (half-life 14 days)
- **Personalization:** boost objects user recently viewed/edited (light touch)
- **Permission filter:** applied as index filter, not post-filter (for correctness + performance)

J.4 Query Understanding

```
text

User: "bugs in payments linked to redesign"
→ {
    type: "bug",
    keywords: ["payments", "redesign"],
    relations: [{ rel: "references", target_keyword: "redesign" }],
    intent: "filter_search"
  }
```

- **Entity extraction:** issue keys, @mentions, #tags, dates
- **Filters:** "assigned to me", "due this week", "in sprint 14"
- **Synonym expansion:** bug ↔ defect ↔ issue; doc ↔ page ↔ spec

J.5 Semantic Search

- **Embedding:** chunk-level for docs (500 tokens), object-level for work items
- **Vector store:** OpenSearch k-NN (or pgvector for smaller tenants)
- **Hybrid query:** BM25 + k-NN → RRF fusion → rerank
- **Freshness:** incremental re-index on event (Kafka consumer, <5s lag)

J.6 Search UX

- **⌘K palette:** instant results (debounced 150ms), grouped by type, recent first
- **Full page:** filters sidebar, result cards with snippets + highlights, saved searches
- **Keyboard:** ↑↓ navigate, Enter open, ⌘Enter open in new tab
- **Empty state:** suggested searches, recent objects
- **Zero results:** "Did you mean...", related objects, create-new CTA

J.7 Performance Targets

Metric	Target
⌘K quick find p95	< 100ms
Full search p95	< 300ms
Semantic search p95	< 500ms
Index freshness	< 5s from mutation
Index size	~1KB/object (text + vector)

J.8 Observability

- Query logs (anonymized) for relevance tuning
- Click-through rate per query type
- Zero-result rate dashboard
- Slow query alerts (>1s)

PART K — Migration & Importers

K.1 Supported Sources

Source	Import scope
Asana	Projects, tasks, subtasks, sections, tags, assignees, due dates, comments, attachments

Source	Import scope
Jira	Projects, issues, epics, sprints, boards, workflows, comments, attachments, custom fields
Notion	Pages, databases, relations, rollups (recomputed), comments, files
Dribbble	Shots, tags, collections (as assets + galleries)
MS Project	Projects, tasks, dependencies, resources, baselines, costs
Trello / Monday / ClickUp	Boards, cards, lists (v2 scope)
CSV / JSON	Generic mapping

K.2 Import Pipeline

```
text

1. CONNECT      OAuth or API key / file upload
2. DISCOVER     Fetch schema + counts (projects, tasks, users, attachments)
3. MAP          Admin maps source fields → Nexus fields (UI wizard)
4. PREVIEW      Dry-run: show 10 sample objects transformed
5. IMPORT       Batched, resumable, rate-limited
6. RECONCILE    Verify counts, flag failures
7. ACTIVATE     Cutover: redirect links, archive source mapping
8. DUAL-RUN     Optional: keep syncing for N weeks
```

K.3 Field Mapping (example: Jira → Nexus)

Jira field	Nexus field	Transform
`summary`	`title`	direct
`description`	`description` (rich text)	ADF → Nexus blocks
`status`	`status`	via workflow map

Jira field	Nexus field	Transform
`assignee`	`assignee_ids`	email → user match
`reporter`	`reporter_id`	email → user match
`issuetype`	`type`	Bug→bug, Story→story, Epic→epic
`sprint`	`sprint_id`	create sprints first
`story_points`	`story_points`	direct
`customfield_10001`	`custom_fields.team`	admin-defined
`attachment`	`assets`	upload to object store
`comment`	`comments`	preserve author + timestamp

Conflict policy: if user match fails, assign to "Import Bot" + flag for review. If field unmapped, store in `custom_fields._imported_{source}`.

K.4 Identity Reconciliation

```
text

1. Match by email (exact)
2. Match by SSO subject (if IdP connected)
3. Fuzzy match by name + domain (flag for review)
4. Unmatched → create "placeholder" user, merge later
```

Placeholder merge: admin UI to merge placeholder → real user (reassigns all objects, preserves history).

K.5 Attachment Handling

- Stream to object storage (S3-compatible)
- Deduplicate by checksum
- Preserve original filename + source URL in metadata
- Virus scan (ClamAV) before marking available
- Size limits per plan (e.g., 100MB free, 5GB enterprise)

K.6 Resumability & Idempotency

- **Checkpointing:** every N objects, save cursor to `import_jobs` table
- **Idempotency key:** `source_id + source_object_id` → skip if already imported
- **Retry:** exponential backoff on transient errors; dead-letter for permanent
- **Resume:** "Continue import" button picks up from last checkpoint

K.7 Cutover & Dual-Run

- **Link rewriting:** old URLs (e.g., `jira.acme.com/browse/NEX-123`) → Nexus deep links via proxy or bookmarklet
- **Dual-run mode:** webhook-based sync from source → Nexus for N weeks (read-only in source)
- **Freeze window:** source made read-only during final sync
- **Rollback:** keep source snapshot; Nexus import is additive (no source mutation)

K.8 Import Observability

- Per-job dashboard: total, imported, skipped, failed, ETA
- Error log with source object links
- Post-import report (counts by type, unmapped fields, identity issues)
- Audit trail of who imported what when

PART L — Full API Reference

L.1 API Surfaces

Surface	Protocol	Use case
GraphQL	POST /graphql	Primary app + integrations
REST	/api/v1/*	Simple CRUD, webhooks, file upload
Webhooks (outbound)	HTTP POST	Event notifications to external systems

Surface	Protocol	Use case
WebSocket	wss://.../realtime	Live updates, presence
Streaming	SSE / gRPC	Large exports, AI streaming

L.2 Authentication

```
text

1. OAuth 2.0 (authorization code + PKCE) for user-facing apps
2. Personal API tokens (scoped) for scripts
3. Service accounts (workspace-scoped) for integrations
4. JWT bearer for internal service-to-service
```

Scopes: `work_item:read`, `work_item:write`, `doc:read`, `doc:write`, `asset:read`, `asset:write`, `admin`, `webhook:manage`.

L.3 GraphQL (extended from Part A)

```graphql
type Subscription {
  workItemUpdated(spaceId: ID!): WorkItem!
  commentAdded(targetId: ID!): Comment!
  presenceChanged(objectId: ID!): PresenceState!
  notificationCreated: Notification!
}

type Mutation {
  # ... existing from Part A ...
  bulkUpdateWorkItems(ids: [ID!]!, input: UpdateWorkItemInput!): BulkResult!
  moveWorkItems(ids: [ID!]!, status: String!): BulkResult!
  createDoc(input: CreateDocInput!): Doc!
  updateDocBlocks(docId: ID!, ops: [BlockOp!]!, version: Int!): Doc!
  uploadAsset(input: UploadAssetInput!): Asset!
  createComment(input: CreateCommentInput!): Comment!
  resolveComment(id: ID!): Comment!
  startApproval(input: StartApprovalInput!): Approval!
  decideApproval(id: ID!, decision: ApprovalDecision!, note: String): Approval!
  runAutomationDryRun(ruleId: ID!, sampleEvent: JSON!): DryRunResult!
  copilotQuery(prompt: String!, context: CopilotContext): CopilotResponse!
}
```

L.4 REST Endpoints (v1)

```
text

# Objects
GET    /api/v1/work_items?filter=...&cursor=...&limit=50
POST   /api/v1/work_items
GET    /api/v1/work_items/{id}
PATCH  /api/v1/work_items/{id}          (If-Match: version)
DELETE /api/v1/work_items/{id}

# Docs
GET    /api/v1/docs/{id}
POST   /api/v1/docs
PATCH  /api/v1/docs/{id}/blocks

# Assets
POST   /api/v1/assets                    (multipart)
GET    /api/v1/assets/{id}/versions
POST   /api/v1/assets/{id}/versions

# Search
GET    /api/v1/search?q=...&type=...&scope=...

# Views
POST   /api/v1/views/project             (server-side ViewSpec projection)
GET    /api/v1/views/{id}

# Automations
GET    /api/v1/automations
POST   /api/v1/automations
POST   /api/v1/automations/{id}/dry_run

# Webhooks (manage outbound)
GET    /api/v1/webhooks
POST   /api/v1/webhooks
DELETE /api/v1/webhooks/{id}

# Admin
GET    /api/v1/workspace/members
POST   /api/v1/workspace/roles
GET    /api/v1/audit?since=...

# Import
POST   /api/v1/imports                   (start job)
```

```
GET    /api/v1/imports/{id}              (status)
POST   /api/v1/imports/{id}/resume
```

L.5 Webhook Contract (outbound)

```
json

POST https://customer.example.com/nexus-webhook
Headers:
  X-Nexus-Signature: sha256=...
  X-Nexus-Event: work_item.status_changed
  X-Nexus-Delivery: uuid
  X-Nexus-Timestamp: 1712345678
Body:
{
  "event_id": "uuid",
  "event": "work_item.status_changed",
  "workspace_id": "uuid",
  "occurred_at": "2026-...",
  "data": {
    "work_item": { "id": "NEX-123", "status": "Done", ... },
    "before": { "status": "In Review" },
    "actor": { "id": "uuid", "name": "Sam" }
  }
}
```

Delivery: at-least-once, retries with exponential backoff (5 attempts over 24h), DLQ after. Signature = HMAC-SHA256 of body with workspace webhook secret. Timestamp window ±5 min to prevent replay.

Events: `work_item.created|updated|deleted|status_changed`, `comment.created`, `approval.decided`, `asset.version_created`, `sprint.started|closed`, `schedule.recalculated`.

L.6 Rate Limits & Quotas

Tier	GraphQL	REST	Webhooks out
Free	100 req/min	60 req/min	10/min
Team	1000 req/min	600 req/min	100/min

Tier	GraphQL	REST	Webhooks out
Enterprise	10k req/min	6k req/min	1000/min

- **429** with `Retry-After` header
- **Cost-based** GraphQL limiting (complex queries cost more)
- Per-token and per-workspace budgets

L.7 Error Format

```json
{
  "error": {
    "code": "FORBIDDEN_FIELD",
    "message": "You cannot edit cost_actual",
    "field": "cost_actual",
    "trace_id": "uuid"
  }
}
```

Codes: `UNAUTHENTICATED`, `FORBIDDEN`, `FORBIDDEN_FIELD`, `NOT_FOUND`, `VALIDATION_ERROR`, `CONFLICT` (with current version), `RATE_LIMITED`, `INTERNAL`.

L.8 SDKs & Tooling

- **TypeScript SDK** (generated from GraphQL schema)
- **Python SDK** (REST + GraphQL)
- **CLI** (`nexus` command: login, import, export, rules)
- **Postman collection** + OpenAPI spec (REST)
- **Webhook testing tool** in admin UI

L.9 Versioning Policy

- **GraphQL:** additive changes only; deprecations with 6-month notice
- **REST:** `/v1` stable; breaking changes → `/v2`
- **Webhooks:** event schema versioned; old events continue during transition

- **Deprecation headers:** `Sunset` , `Deprecation`

PART M — Visual Wireframes

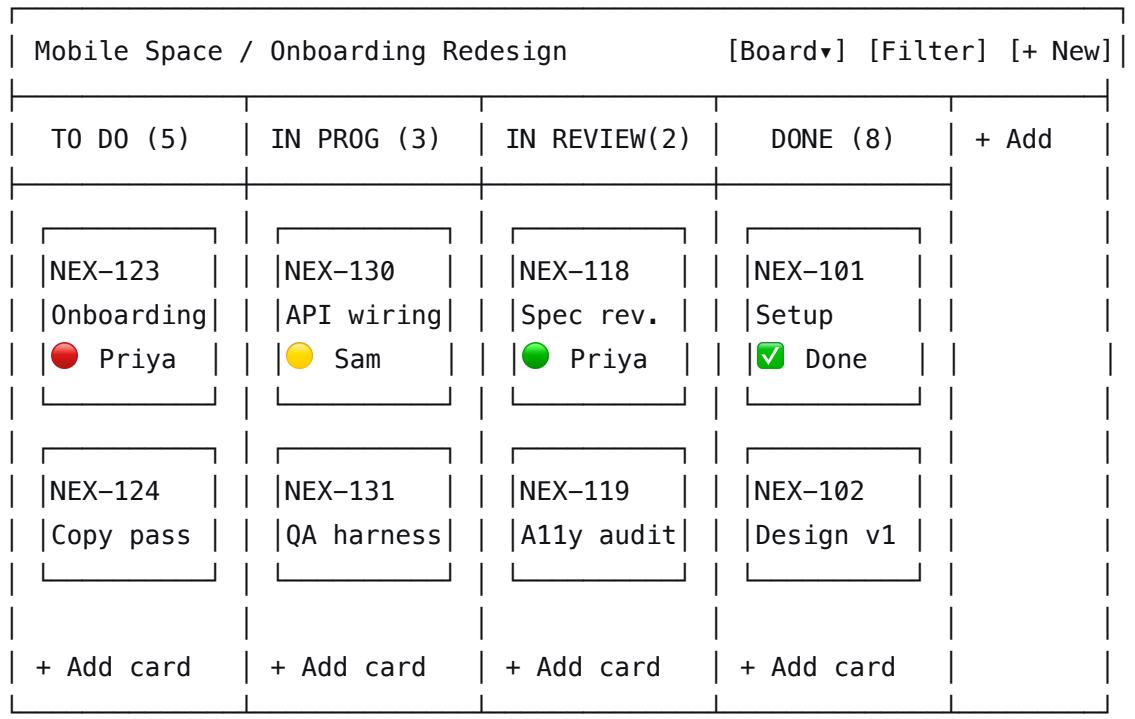
M.1 Dashboard Home (post-login)

text

```
┌──────────────────────────────────────────────────────────────────────┐
│ ≡  Nexus          [🔍 Search or ⌘K]          🔔3   👤 Priya ▾      │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  Good morning, Priya 👋                                              │
│  You have 5 tasks due today · 2 overdue · 1 approval waiting         │
│                                                                      │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐                 │
│  │ DUE TODAY│ │ OVERDUE  │ │IN REVIEW │ │APPROVALS │                 │
│  │    5     │ │    2 🔴  │ │    4     │ │    1 ⏳  │                 │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘                 │
│                                                                      │
│  MY WORK                                    [Filter ▾] [+ New]       │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ □ NEX-123  Onboarding flow redesign     🔴 Overdue · Today   │    │
│  │ □ NEX-118  Payments v2 spec review      🟡 Due tomorrow      │    │
│  │ □ NEX-140  Fix login crash (Sprint 14)  🟢 In Review         │    │
│  │ □ SHOT-88  Homepage v3 — 2 new comments 💬                   │    │
│  │ □ APR-12   Q3 budget baseline change    ⏳ Your sign-off     │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────┐  ┌──────────────────────────┐          │
│  │ TODAY'S SCHEDULE         │  │ MENTIONS & ACTIVITY      │          │
│  │ 10:00 Sprint review      │  │ @you in NEX-123          │          │
│  │ 14:00 Design sync        │  │ @you in "Spec: Onboard"  │          │
│  │ 16:00 1:1 with Sam       │  │ Sam approved your shot   │          │
│  └──────────────────────────┘  └──────────────────────────┘          │
│                                                                      │
│  ┌──────────────────────────┐  ┌──────────────────────────┐          │
│  │ MY SPRINT (14)           │  │ INITIATIVES I OWN        │          │
│  │ ▓▓▓
```

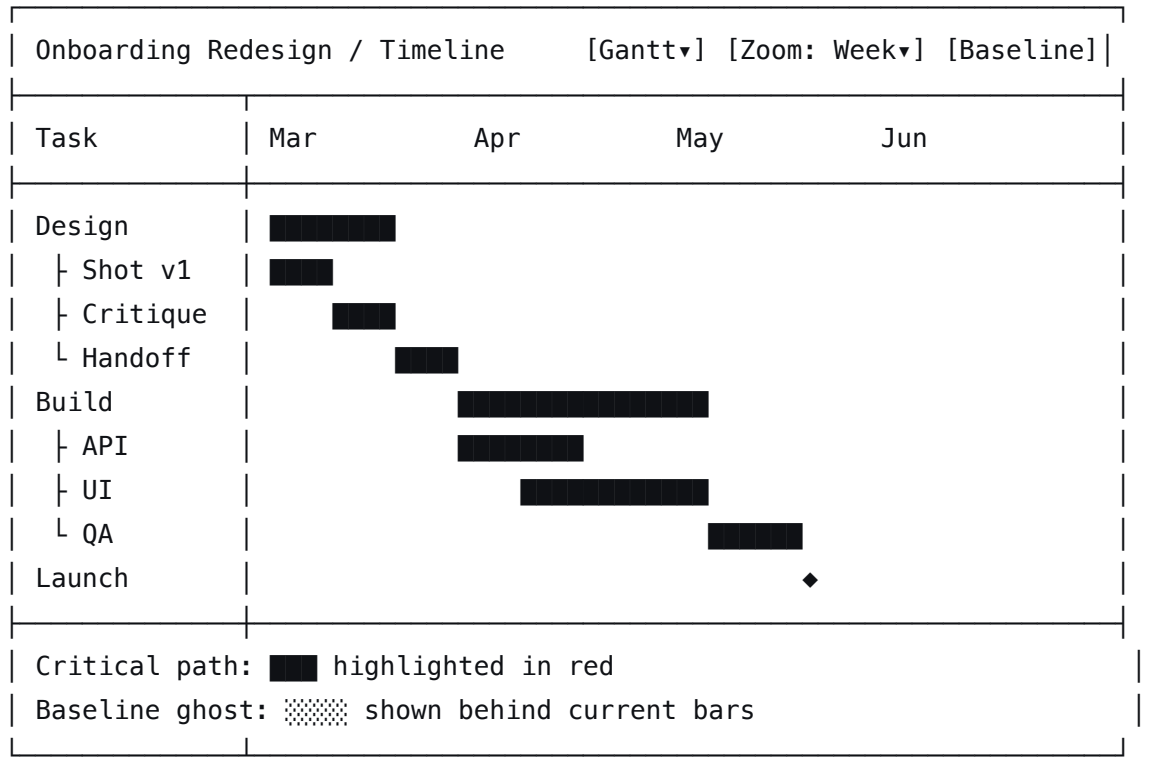
M.2 Board View

text



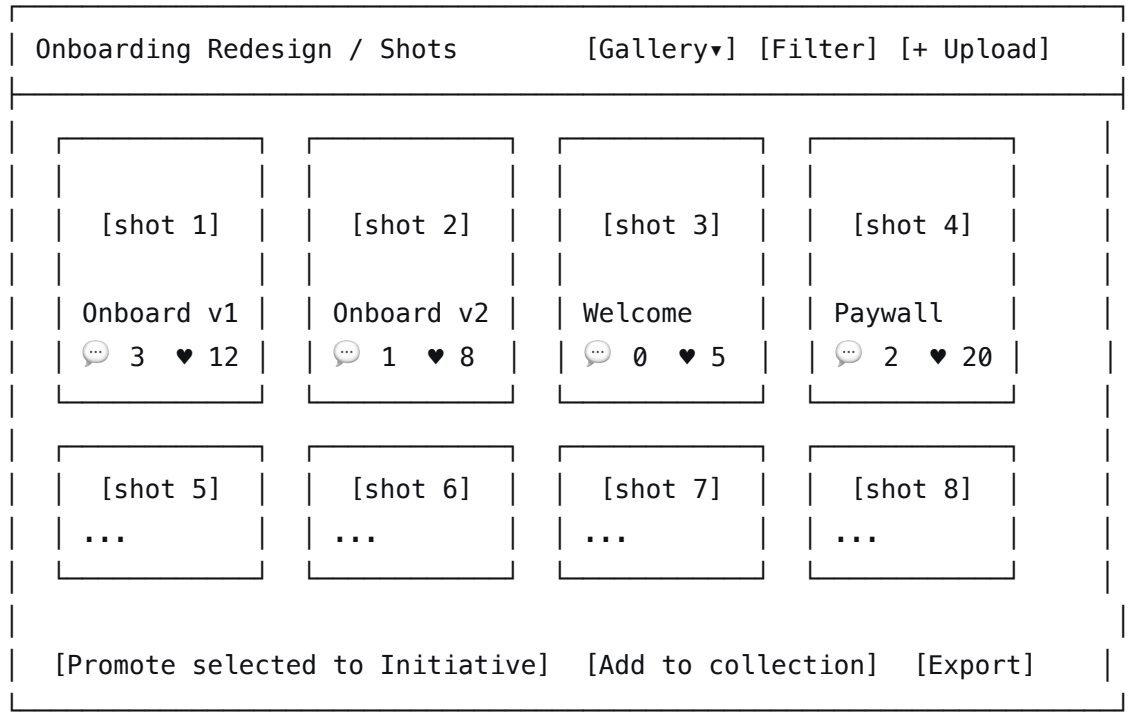
M.3 Gantt View

text



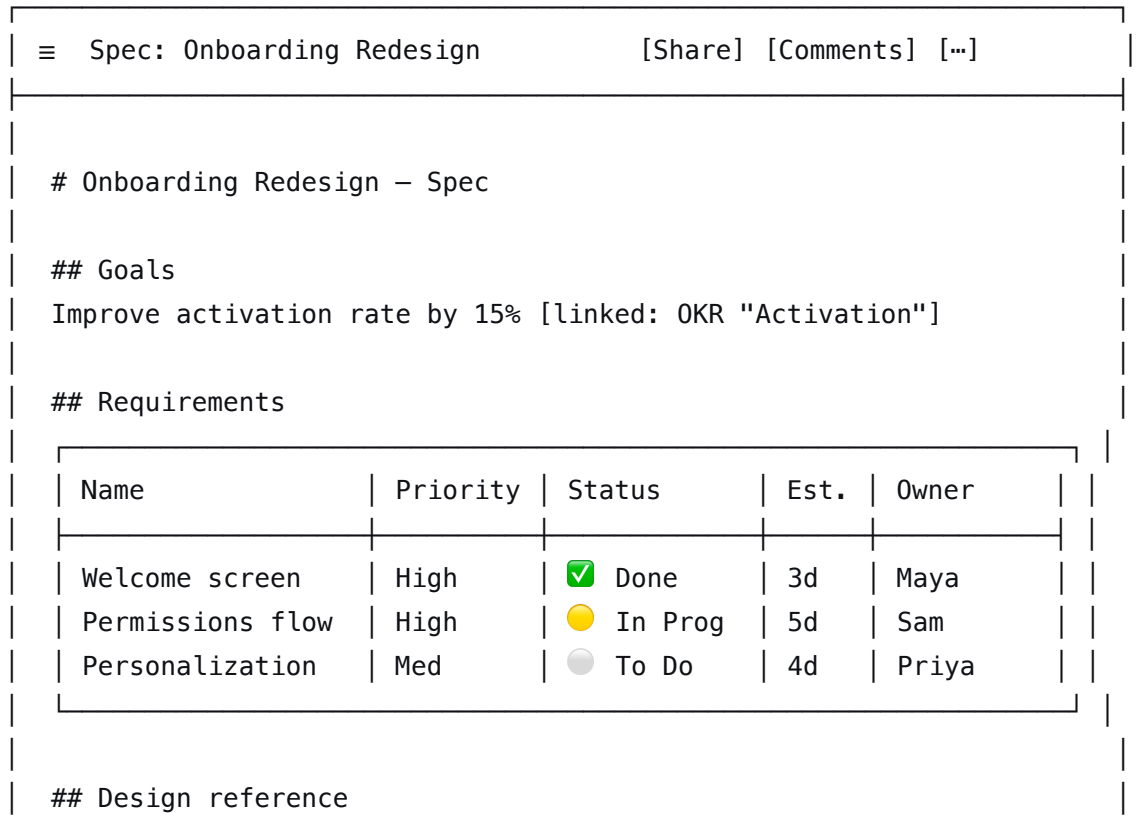
M.4 Gallery View

text



M.5 Doc View

text



```
│  [embedded shot: Onboard v2]  ← live link to asset                  │
│                                                                     │
│  ## Open questions                                                  │
│  - Do we need A/B test? @sam?  ← inline comment thread              │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

M.6 Copilot Sidebar

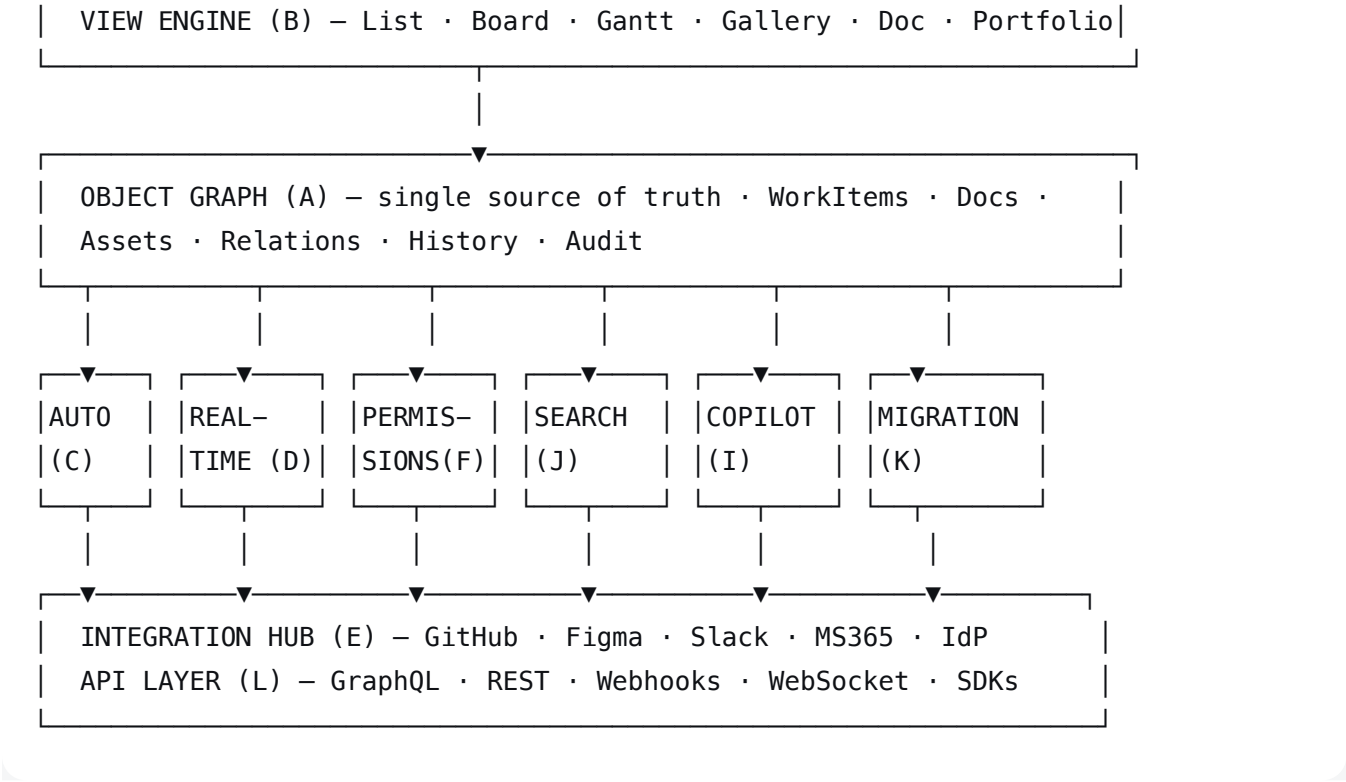
text

```
┌──────────────────────────────────────┐
│  ✨ Copilot                   [×]  │
├──────────────────────────────────────┤
│  You: Summarize this initiative     │
│                                      │
│  Copilot:                            │
│  Onboarding Redesign is 78% done.   │
│  Design is complete [1]. Build is   │
│  in progress with 2 risks: API      │
│  dependency slip [2] and QA         │
│  capacity [3].                       │
│                                      │
│  [1] Doc "Spec: Onboarding"         │
│  [2] NEX-130 · comment by Sam       │
│  [3] Sprint 14 · capacity view      │
│                                      │
│  ──────────────────────────      │
│  [Draft status update] [Ask more]   │
├──────────────────────────────────────┤
│  [Type a message…]           [↑]    │
└──────────────────────────────────────┘
```

Summary — Complete System Map (Parts A–M)

text

```
┌─────────────────────────────────────────────────────────────────────┐
│  DASHBOARD HOME (G) — first screen after login                      │
│  All my tasks · across all layers · permission-filtered · live      │
└──────────────────────────┬──────────────────────────────────────────┘
                           │
┌──────────────────────────▼──────────────────────────────────────────┐
```



The dashboard (G) is the front door; the object graph (A) is the foundation; everything else is a lens (B), an actor (C), a channel (D), a gate (F), a finder (J), an assistant (I), a bridge (K/E), or a doorway (L).

Would you like me to continue with:

- **Part N: Testing & QA strategy** (unit, integration, E2E, load, security)
- **Part O: Deployment & infrastructure** (K8s, multi-region, DR, cost model)
- **Part P: Analytics & product metrics** (what to instrument, dashboards)
- **Part Q: Pricing & packaging** (tiers, feature gating, usage metering)
- Or a **consolidated master table of contents** tying all parts A–M into one navigable document?

yes proceed

Technical Specification Expansion — Parts N–Q + Master Index

Continuing from Parts A–M. This document covers:

- **Part N:** Testing & QA strategy

- **Part O:** Deployment & infrastructure (K8s, multi-region, DR, cost)
- **Part P:** Analytics & product metrics
- **Part Q:** Pricing & packaging
- **Part R:** Consolidated master index (all parts A–Q)

PART N — Testing & QA Strategy

N.1 Test Pyramid

```
text

          ┌──────────────┐
          │   E2E (5%)   │  Playwright · full user journeys
          ├──────────────┤
          │ Integ (25%)  │  API + DB + Kafka + connectors
          ├──────────────┤
          │ Unit (70%)   │  Pure logic, resolvers, DSL, CRDT
          └──────────────┘
```

N.2 Test Categories

Layer	Tool	Scope	Gate
Unit	Vitest / Jest	Functions, resolvers, expression evaluator, CRDT merge	Every PR, 80% coverage on core
Contract	Pact / GraphQL schema diff	API ↔ client, connector ↔ hub	Every PR
Integration	Testcontainers (Postgres, Redis, Kafka)	Object graph + automations + views	Every PR
E2E	Playwright	Login → dashboard → create → board → Gantt → copilot	Pre-merge + nightly

Layer	Tool	Scope	Gate
Visual	Chromatic / Percy	All layouts, responsive breakpoints	Every PR (component)
Load	k6 / Gatling	10k concurrent, 100k objects/workspace	Weekly + pre-release
Chaos	Toxiproxy / Litmus	Kafka down, Redis down, connector failure	Monthly
Security	SAST (Semgrep), DAST (ZAP), SCA (Snyk)	OWASP Top 10, dependency CVEs	Every PR + nightly
Accessibility	axe-core + manual	WCAG 2.1 AA across all views	Every PR
AI eval	Golden set (200 Q&A)	Citation accuracy, groundedness	Every copilot change
Migration	Fixture imports from 5 tools	Field mapping, identity, attachments	Every importer change

N.3 Critical Test Scenarios (must-pass)

Object Graph (A)

- Concurrent edit → optimistic concurrency `CONFLICT` with current version
- Relation cycle detection (depends_on loops rejected)
- Field-level mask never leaks via GraphQL, REST, export, or search
- Audit log immutable (trigger prevents UPDATE/DELETE)

View Engine (B)

- Same dataset renders correctly in all 8 layouts
- Server projection matches client projection (parity test)
- Gantt with 10k items renders < 300ms (virtualized window)

- ViewSpec validation rejects incompatible renderer

Automation DSL (C)

- Loop protection halts at depth 5
- Idempotency: same event twice → one action
- Dry-run produces same result as live run
- Permission: rule cannot exceed creator's grants

Real-time (D)

- CRDT merge: two clients edit same paragraph → both preserved
- Reconnect with `lastEventId` replays missed events
- Presence TTL expires after 60s
- Degraded mode (polling) keeps data consistent

Permissions (F)

- Guest sees only granted objects (query-level filter)
- Field mask enforced in UI, API, export, search, copilot retrieval
- Role change propagates within 5s (cache invalidation)
- Service principal cannot escalate

Dashboard (G)

- Aggregates from all 5 layers correctly
- Permission-filtered (no cross-space leakage)
- Loads < 1.5s first paint; widgets lazy-load
- Offline shows cached with banner

Copilot (I)

- Citation accuracy ≥ 95% on golden set
- Ungrounded claims flagged
- Prompt injection attempts blocked
- Cross-tenant retrieval impossible

Search (J)

- ⌘K p95 < 100ms
- Permission filter applied as index filter (not post-filter)
- Semantic + keyword fusion returns expected top-5 on golden queries

Migration (K)

- 10k-object Jira import completes without data loss
- Identity reconciliation matches ≥ 90% by email
- Resume from checkpoint works
- Rollback leaves source untouched

API (L)

- Rate limit returns 429 with `Retry-After`
- Webhook signature validates; replay window ±5 min
- GraphQL cost limiter blocks expensive queries
- Deprecation headers present on sunset fields

N.4 Test Data & Environments

Env	Purpose	Data
Local	Dev	Seeded demo workspace (script)
CI	Automated	Ephemeral per-PR (Testcontainers)
Staging	Pre-prod	Anonymized prod snapshot (weekly)
Prod	Live	Real

Seed script creates: 1 workspace, 3 spaces, 50 work items, 10 docs, 20 assets, 2 sprints, 1 initiative with Gantt, 5 automation rules, 3 users with different roles.

N.5 Release Gates

Gate	Criteria
PR merge	Unit + integration + contract + visual + a11y pass

Gate	Criteria
Staging deploy	E2E pass, no P0/P1 bugs open
Prod release	Load test pass, security scan clean, canary 5% for 24h
Copilot release	Golden set citation accuracy ≥ 95% (no regression)
Importer release	Fixture imports pass for all 5 sources

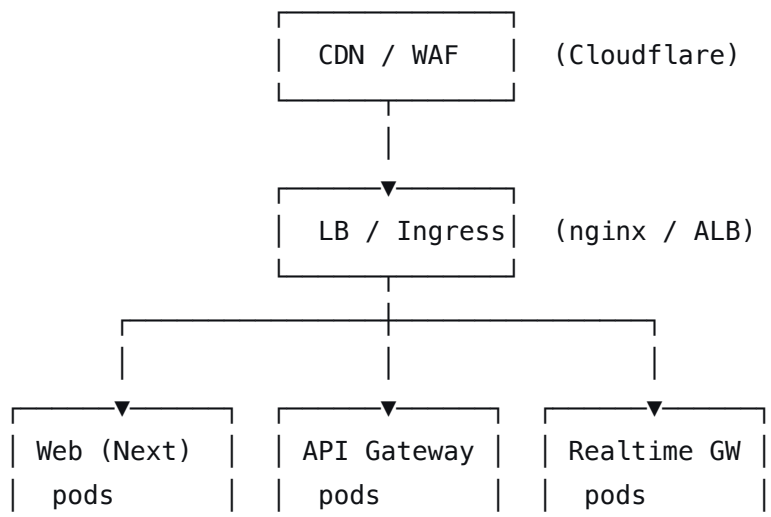
N.6 Bug Severity & SLA

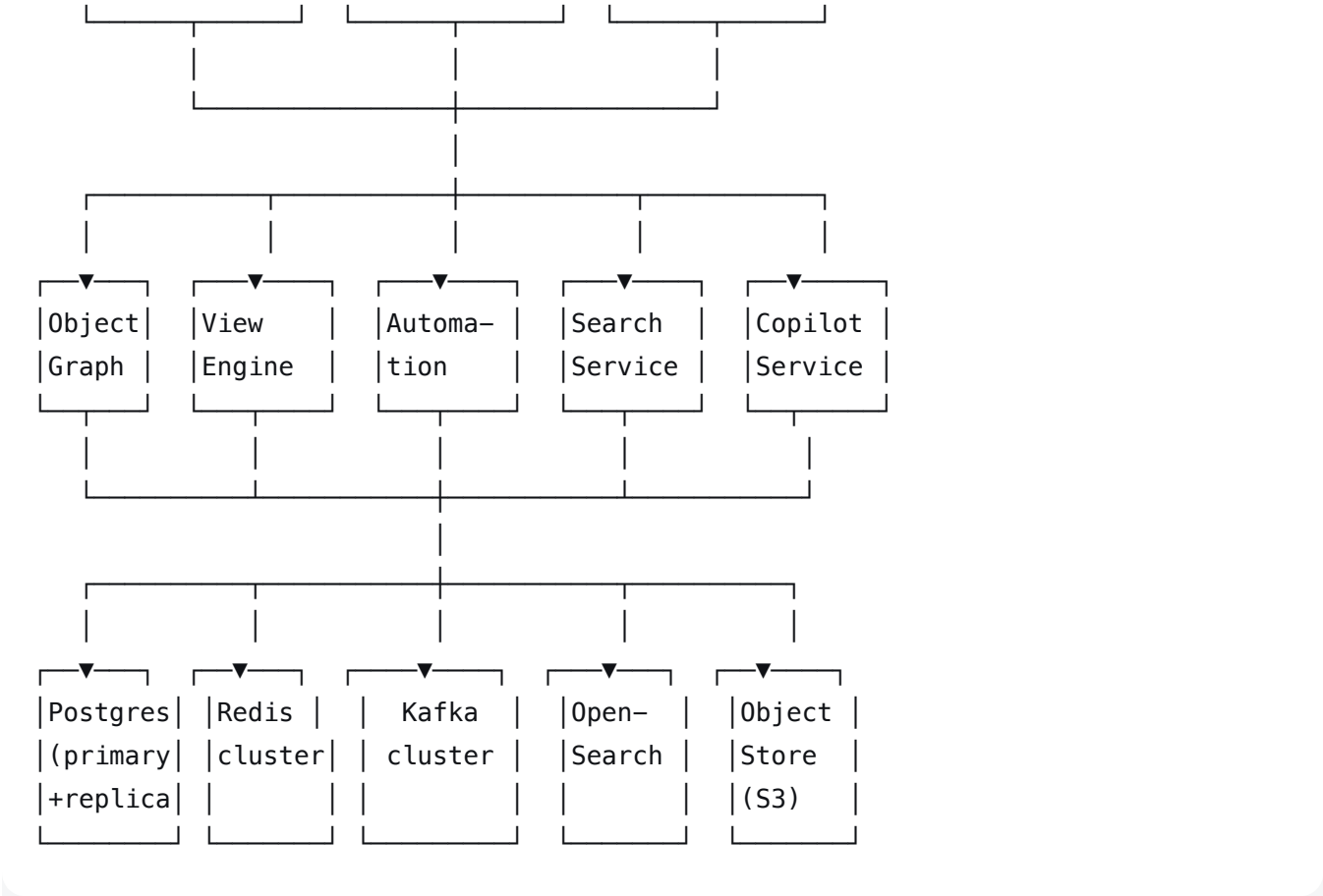
Severity	Definition	Fix SLA
P0	Data loss, security breach, outage	4h
P1	Core flow broken, no workaround	24h
P2	Feature broken, workaround exists	1 week
P3	Cosmetic, minor	Next sprint

PART O — Deployment & Infrastructure

O.1 Architecture (production)

text





O.2 Kubernetes Layout

Namespace	Workloads
`nexus-web`	Next.js frontend, static assets via CDN
`nexus-api`	API Gateway, GraphQL, REST
`nexus-realtime`	WebSocket gateways (sticky)
`nexus-services`	Object Graph, View Engine, Automation, Search, Copilot
`nexus-workers`	Kafka consumers, importer, scheduler, notification
`nexus-data`	Postgres (operator), Redis, Kafka, OpenSearch
`nexus-observability`	Prometheus, Grafana, Loki, Tempo

Scaling:

- HPA on CPU + custom metrics (queue depth, WS connections)

- Realtime gateways scale by connection count (target 5k/pod)
- Workers scale by Kafka lag

O.3 Multi-Region Strategy

Tier	Strategy
Free/Team	Single region (us-east-1)
Enterprise	Data residency choice (us, eu, apac)
Global read	CDN for static; read replicas per region for Postgres
Write	Primary region; cross-region async replication
Search	Per-region index, replicated from primary events
Object storage	Regional buckets; cross-region replication for DR

Tenant routing: workspace → home region stored in `workspaces.region`; API gateway routes accordingly.

O.4 Disaster Recovery

Metric	Target
RPO (recovery point)	5 min (Postgres WAL streaming)
RTO (recovery time)	1 hour (failover to replica region)
Backup	Postgres PITR (7 days), daily snapshots (30 days)
Object store	Cross-region replication
Kafka	MirrorMaker to DR cluster
Runbook	Documented failover + failback; quarterly drill

O.5 CI/CD Pipeline

```
text

PR → lint → unit → integration → contract → visual → a11y → security
   → build image → deploy preview env → E2E on preview
   → merge to main → deploy staging → E2E + load smoke
   → canary 5% prod (24h) → full rollout
```

Tools: GitHub Actions, ArgoCD (GitOps), Terraform (infra), Helm charts.

Rollback: ArgoCD one-click revert; DB migrations backward-compatible (expand/contract pattern).

O.6 Observability

Signal	Tool	Key metrics
Metrics	Prometheus + Grafana	Request rate, latency p50/p95/p99, error rate, saturation
Logs	Loki	Structured JSON, trace_id correlation
Traces	Tempo (OpenTelemetry)	End-to-end spans: API → service → DB → event
Alerts	Alertmanager → PagerDuty	SLO burn rate, error budget
RUM	Sentry / custom	Core Web Vitals, JS errors
AI	Custom	Copilot latency, citation accuracy, token spend

SLOs:

- API availability 99.95%
- API p95 latency < 300ms
- Realtime message delivery < 500ms p95
- Search p95 < 300ms
- Dashboard first paint < 1.5s

O.7 Security Infrastructure

- **Secrets:** Vault / cloud KMS; no secrets in env or code
- **Encryption:** TLS 1.3 in transit; AES-256 at rest (DB, object store)
- **Network:** private subnets for data; service mesh (mTLS) between services
- **WAF:** OWASP rules, rate limiting, bot detection
- **DDoS:** Cloudflare / cloud provider protection
- **Vulnerability scanning:** daily image scans (Trivy), dependency SCA
- **Penetration testing:** annual third-party + quarterly internal

O.8 Cost Model (rough, per 1k active users/month)

Component	Est. cost
Compute (K8s)	$2–4k
Postgres (HA)	$1–2k
Redis	$300–600
Kafka	$500–1k
OpenSearch	$800–1.5k
Object storage	$200–500
Copilot LLM tokens	$500–2k (usage-dependent)
CDN + WAF	$200–500
Observability	$500–1k
Total	**~$6–13k / 1k users**

Optimization levers: embedding cache, LLM response cache, tiered storage (cold assets), reserved instances.

PART P — Analytics & Product Metrics

P.1 Analytics Architecture

```
text

Client events ──► Event Collector (Kafka) ──► Warehouse (ClickHouse/BigQuery)
                                                        │
Server events ──► (same pipeline)                       ▼
                                                  dbt models → BI dashboards
```

- **Client events:** page views, clicks, view switches, search queries, copilot usage
- **Server events:** object mutations, automation runs, integration syncs, API calls
- **Identity:** `user_id` , `workspace_id` , `plan` , `role`
- **Privacy:** PII excluded; user opt-out; GDPR delete propagates to warehouse

P.2 North Star & Key Metrics

North Star: Weekly Active Collaborators (WAC) — users who create/edit/comment on ≥1 object in a week.

Category	Metric	Target
Activation	% new users who complete first task in 24h	> 60%
Engagement	DAU/WAU	> 40%
Retention	W4 retention	> 50%
Depth	Avg objects touched/user/week	> 10
Cross-layer	% users active in ≥3 layers	> 30% (the "marriage" proof)
Collaboration	% objects with ≥2 collaborators	> 50%
Dashboard	% sessions starting at dashboard	> 70%
Search	⌘K usage per user/day	> 5
Copilot	% users using copilot weekly	> 25%
Automation	Rules per workspace	> 5

Category	Metric	Target
Integration	Connected integrations per workspace	> 2
Expansion	Net revenue retention	> 120%

P.3 Funnel Metrics

```
text

Signup → Workspace created → First space → First work item
  → First view switch → First comment → First automation
  → First integration → Invite teammate → Paid conversion
```

Each step instrumented; drop-off dashboard; A/B tests on onboarding.

P.4 Layer Adoption (the "marriage" scorecard)

Layer	Metric
Visual	Assets uploaded/week, critique comments/week
Work Mgmt	Tasks created/week, views switched
Agile	Sprints active, stories completed
Knowledge	Docs created, database rows
Portfolio	Initiatives tracked, Gantt edits

Cross-layer health: % of work items linked to ≥2 layers (e.g., task with asset + doc + sprint).

P.5 Product Dashboards

Dashboard	Audience	Contents
Executive	Leadership	WAC, NRR, churn, activation

Dashboard	Audience	Contents
Product	PM	Feature adoption, funnel, retention cohorts
Engineering	Eng	SLOs, error rates, latency, deploy frequency
Copilot	AI team	Usage, citation accuracy, token spend, thumbs up/down
Integration	Platform	Connector health, sync lag, errors
Support	CS	Ticket volume, top issues, at-risk accounts

P.6 Experimentation

- **Feature flags:** LaunchDarkly / Unleash; per-workspace targeting
- **A/B tests:** sample size calculator; guardrail metrics (latency, errors)
- **Holdout groups:** 5% never see new features (long-term impact measurement)

P.7 Instrumentation Standards

```typescript
track('work_item.created', {
  workspace_id, user_id, plan, role,
  work_item_type, space_id, source: 'board'|'list'|'api',
  has_asset: boolean, has_doc: boolean,
});
```

- Naming: `object.action` (snake_case)
- Required props: `workspace_id` , `user_id` , `plan`
- No PII in event props
- Schema registry validates events

PART Q — Pricing & Packaging

Q.1 Tiers

	Free	Team	Business	Enterprise
Price	$0	$12/user/mo	$24/user/mo	Custom
Users	10	Unlimited	Unlimited	Unlimited
Objects	5k	100k	1M	Unlimited
Storage	1GB	100GB	1TB	Custom
Layers	Tasks + Docs + Board	+ Sprints + Gallery	+ Gantt + Portfolio	All
Views	List, Board, Doc	+ Gallery, Calendar	+ Gantt, Timeline	+ Portfolio, custom
Automations	5 rules	50 rules	500 rules	Unlimited
Integrations	2	10	All + webhooks	All + custom
Copilot	50 queries/mo	500/mo	5k/mo	Unlimited*
Search	Basic	Hybrid	Hybrid + semantic	+ custom ranking
Permissions	Basic roles	+ Space admin	+ Field-level	+ Custom roles
SSO	—	Google	Google + SAML	+ SCIM
Audit log	—	30 days	1 year	7 years
Support	Community	Email	Priority	Dedicated CSM + SLA
Data residency	—	—	—	✅
Uptime SLA	—	—	99.9%	99.95%

*Fair-use cap; overage billed.

Q.2 Feature Gating Matrix (key gates)

Feature	Free	Team	Business	Enterprise
Gantt view	❌	❌	✅	✅
Critical path	❌	❌	✅	✅
Portfolio dashboard	❌	❌	✅	✅
Resource leveling	❌	❌	✅	✅
Scenario planning	❌	❌	❌	✅
Field-level permissions	❌	❌	✅	✅
Custom workflows	❌	✅	✅	✅
Automation DSL	basic	standard	advanced	+ webhooks
Copilot tool use	❌	❌	✅	✅
Migration importers	CSV	+ Asana/Jira	+ all	+ custom
API rate limit	100/min	1k/min	10k/min	custom

Q.3 Usage Metering

Meter	Unit	Billing
Objects	count	Tier cap
Storage	GB	Overage $0.10/GB
Copilot	queries	Overage $0.02/query

Meter	Unit	Billing
API calls	requests	Overage per 10k
Automation runs	executions	Overage per 1k
Integration syncs	events	Tier cap

Metering pipeline: events → usage aggregator → billing service → Stripe.

Q.4 Packaging Principles

1. **Land with one layer, expand to five** — Free tier gives tasks+docs; Gantt/portfolio unlock at Business (where PMOs live)
2. **Copilot as expansion lever** — generous Team, metered Business
3. **Enterprise gates on governance** — SSO, audit, residency, custom roles
4. **No per-feature micro-transactions** — tiers, not add-ons (except enterprise custom)
5. **Non-profit / education discount** — 50%

Q.5 Trials & Conversion

- **14-day Business trial** on signup (no card)
- **In-product upgrade prompts** at cap boundaries (objects, automations, copilot)
- **CSM-assisted** for >50-user trials
- **Conversion funnel:** trial start → day 3 (activation) → day 7 (integration) → day 14 (conversion)

Q.6 Revenue Metrics

Metric	Definition
MRR	Monthly recurring revenue
ARR	Annualized
NRR	Net revenue retention (expansion – churn)
Gross margin	(Revenue – infra – LLM) / Revenue

Metric	Definition
CAC payback	Months to recover acquisition cost
Logo churn	% accounts lost/month

PART R — Consolidated Master Index

R.1 Document Map (A–Q)

Part	Title	Key contents
—	Design Doc v1	Vision, architecture, layers, personas, roadmap
A	Object Graph Schema	Tables, GraphQL, events, storage strategy
B	View Engine Contract	ViewSpec, renderers, layouts, server projection
C	Automation DSL	Grammar, actions, expressions, execution, examples
D	Real-Time Collaboration	CRDT (Yjs), presence, conflict UX, fan-out
E	Integration Connectors	Field ownership, OAuth, webhooks, catalog
F	Permissions & RBAC	Roles, field-level masks, inheritance, audit
G	Dashboard Home	Post-login "all my tasks" hub, data sources, personalization
H	Developer Onboarding	First-week plan, day-by-day
I	AI Copilot	Retrieval, grounding, citations, guardrails, tools

Part	Title	Key contents
J	Search Architecture	Hybrid ranking, index design, UX, performance
K	Migration & Importers	Pipeline, field mapping, identity, cutover
L	Full API Reference	GraphQL, REST, webhooks, auth, rate limits, SDKs
M	Visual Wireframes	Dashboard, Board, Gantt, Gallery, Doc, Copilot
N	Testing & QA	Pyramid, scenarios, gates, SLA
O	Deployment & Infra	K8s, multi-region, DR, CI/CD, observability, cost
P	Analytics & Metrics	North star, funnels, dashboards, instrumentation
Q	Pricing & Packaging	Tiers, gating, metering, trials, revenue metrics

R.2 Reading Paths

For leadership: Design Doc → Q (pricing) → P (metrics) → O (cost/DR)
For product: Design Doc → G (dashboard) → B (views) → P (metrics) → Q
For engineering: A → B → C → D → E → F → L → N → O
For AI/data: A → J → I → P
For GTM/CS: Design Doc → G → Q → K (migration) → P

R.3 Cross-Reference: Feature → Parts

Feature	Parts
Unified object (task=issue=card=row=bar)	A, B
Multi-view (list/board/gantt/gallery/doc)	B, M

Feature	Parts
Dashboard home (all my tasks)	G, M
Automations	C
Real-time collab	D
Integrations	E, L
Permissions	F
Copilot	I
Search	J
Migration	K
API	L
Testing	N
Infra	O
Metrics	P
Pricing	Q

R.4 Open Decisions (to resolve before build)

#	Decision	Owner	Impact
1	Layer toggling per workspace (hide Gantt for small teams)?	Product	UX, pricing
2	Offline depth for Gantt editing	Eng	Complexity
3	LLM provider (hosted vs self-hosted default)	AI/Eng	Cost, residency
4	CRDT scope: docs only, or also board/Gantt?	Eng	Conflict UX

#	Decision	Owner	Impact
5	Pricing: per-seat vs per-layer vs usage hybrid	GTM	Revenue model
6	Region strategy for Team tier (single vs multi)	Infra	Cost, latency
7	Migration dual-run duration (2 vs 4 weeks)	CS	Adoption
8	Copilot fair-use cap for Enterprise	GTM	Margin

R.5 Glossary

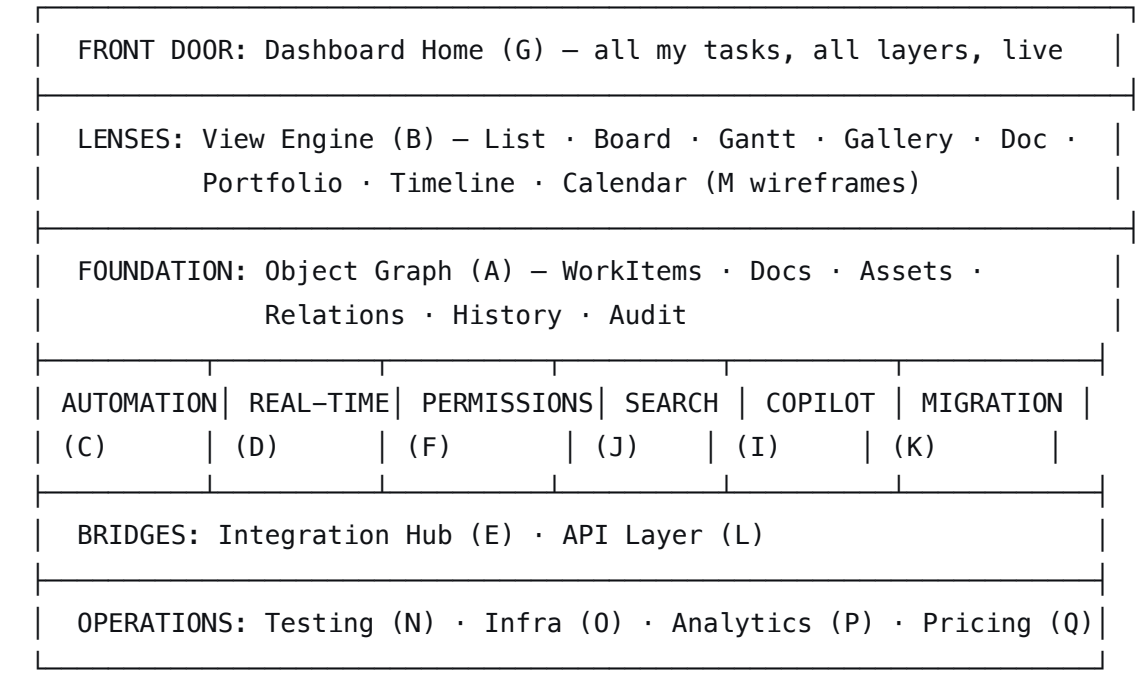
Term	Definition
WorkItem	Universal object: task = issue = card = DB row = Gantt bar
ViewSpec	Declarative lens: dataset + layout + fields + interactions
Lens	A rendered view over the object graph
Initiative	Portfolio-level work item (MS Project project)
Space	Container for work (team/project area)
Relation	Typed edge between objects (depends_on, blocks, etc.)
Field mask	Field-level permission (hidden/readonly/editable)
Rule	Automation in the DSL
Connector	Integration implementation
Projection	Server-shaped RenderModel for a view
WAC	Weekly Active Collaborators (north star)

R.6 Change Log

Version	Date	Changes
1.0	—	Design Doc (vision, architecture, layers)
1.1	—	Parts A–C (object graph, view engine, automation)
1.2	—	Parts D–H (real-time, integrations, permissions, dashboard, onboarding)
1.3	—	Parts I–M (copilot, search, migration, API, wireframes)
1.4	—	Parts N–Q (testing, infra, analytics, pricing) + master index

Final Summary — The Complete System

text



The full spec now covers: vision → data model → views → automation → collaboration → integrations → permissions → the post-login dashboard → AI → search → migration → API → wireframes → testing → deployment → metrics → pricing — with a master index tying it all together.

This is a build-ready design documentation set. The natural next steps would be:

1. **Resolve the 8 open decisions** (R.4) with stakeholders
2. **Prioritize the roadmap** (Design Doc §11) into sprint-sized epics
3. **Staff the first phase** (P1: object graph + view engine + docs + tasks + board)
4. **Stand up the dev environment** (H.1) and start on Day 1 of the onboarding guide

Would you like me to produce any of these next artifacts — e.g., a **sprint-by-sprint P1**

Select all | 7 conversation turns selected Create public link